Cargo.toml

```
[package]
name = "QuTY"
version = "0.1.0"
edition = "2021"

[dependencies]
num-complex = "0.4"
rand = "0.8"
serde = { version = "1.0", features = ["derive"] }
serde_json = "1.0"
eframe = "0.27"
```

main.rs

```
mod physical_specs;
mod channels;
mod density_matrix;
mod qasm;
mod circuit_grid;
mod emitters;
mod postprocessing;
mod app;

use app::TransmonApp;
use eframe::egui;

fn main() -> eframe::Result<()> {
    let native_options = eframe::NativeOptions {
        viewport: egui::ViewportBuilder::default()
            .with_inner_size([1180.0, 760.0])
            .with_min_inner_size([740.0, 480.0])
            .with_title("Qty - Quantum Transmon & Yield Console"),
        ..Default::default()
    };

    eframe::run_native(
        "Qty - Quantum Transmon & Yield Console",
        native_options,
        Box::new(|_cc| Box::new(TransmonApp::default())),
    )
}
```

qasm.rs

```
use crate::physical_specs::{PhysicalIonTrapSpec, PhysicalTransmonSpec};
use num_complex::Complex64;
use rand::Rng;
use std::collections::HashMap;

#[derive(Debug, Clone)]
pub struct QasmProgram {
    pub num_qubits: usize,
    pub instructions: Vec<String>,
}

impl QasmProgram {
```

```
pub fn from_str(qasm: &str) -> Result<Self, String> {
    let mut num_qubits = 3;
    let mut instructions = Vec::new();

    for line in qasm.lines().map(str::trim) {
        if line.starts_with("qreg") {
            if let (Some(s), Some(e)) = (line.find('['), line.find(']')) {
                if let Ok(n) = line[s + 1..e].parse::<usize>() {
                    num_qubits = n.clamp(1, 16);
                }
            }
        } else if !line.is_empty()
            && !line.starts_with("//")
            && !line.starts_with("OPENQASM")
            && !line.starts_with("include")
            && !line.starts_with("creg")
        {
            instructions.push(line.to_string());
        }
    }

    Ok(Self {
        num_qubits,
        instructions,
    })
}

pub fn simulate_ideal(&self) -> Vec<Complex64> {
    let n = self.num_qubits;
    let dim = 1 << n;
    let mut state = vec![Complex64::new(0.0, 0.0); dim];
    if dim > 0 {
        state[0] = Complex64::new(1.0, 0.0);
    }

    for inst in &self.instructions {
        Self::apply_instruction(&mut state, n, inst);
    }

    state
}

pub fn run_noisy_simulation(
    &self,
    spec: &PhysicalTransmonSpec,
    shots: usize,
) -> HashMap<String, usize> {
    let mut results = HashMap::new();
    if shots == 0 {
        return results;
    }

    let ideal_state = self.simulate_ideal();
    let dim = 1 << self.num_qubits;
    let mut probs: Vec<f64> = ideal_state.iter().map(|c| c.norm_sqr()).collect();

    let total_prob: f64 = probs.iter().sum();
    if total_prob > 0.0 {
```

```
        for p in &mut probs {
            *p /= total_prob;
        }
    } else if dim > 0 {
        probs[0] = 1.0;
    }

    let decoherence_rate = ((spec.gate_time_1q_us / spec.t1_us)
        + (spec.gate_time_1q_us / spec.t2_us))
        .clamp(0.0, 0.5);

    let mut cum_probs = Vec::with_capacity(dim);
    let mut acc = 0.0;
    for &p in &probs {
        acc += p;
        cum_probs.push(acc);
    }

    let mut rng = rand::thread_rng();

    for _ in 0..shots {
        let mut val = if rng.gen::<f64>() >= decoherence_rate {
            let r: f64 = rng.gen();
            let mut chosen = 0;
            for (idx, &cp) in cum_probs.iter().enumerate() {
                if r <= cp {
                    chosen = idx;
                    break;
                }
            }
            chosen
        } else {
            rng.gen_range(0..dim)
        };

        for q in 0..self.num_qubits {
            let bit = (val >> q) & 1;
            if bit == 0 {
                if rng.gen::<f64>() < spec.readout_p01 {
                    val ^= 1 << q;
                }
            } else if rng.gen::<f64>() < spec.readout_p10 {
                val ^= 1 << q;
            }
        }

        let bitstring = format!("{:0width$b}", val, width = self.num_qubits);
        *results.entry(bitstring).or_insert(0) += 1;
    }

    results
}

pub fn run_noisy_simulation_ion(
    &self,
    spec: &PhysicalIonTrapSpec,
    shots: usize,
) -> HashMap<String, usize> {
```

```
let mut results = HashMap::new();
if shots == 0 {
    return results;
}

let ideal_state = self.simulate_ideal();
let dim = 1 << self.num_qubits;
let mut probs: Vec<f64> = ideal_state.iter().map(|c| c.norm_sqr()).collect();

let total_prob: f64 = probs.iter().sum();
if total_prob > 0.0 {
    for p in &mut probs {
        *p /= total_prob;
    }
} else if dim > 0 {
    probs[0] = 1.0;
}

let mut n_1q = 0;
let mut n_2q = 0;
for inst in &self.instructions {
    let cl = inst.trim();
    if cl.starts_with("cx ") || cl.starts_with("cnot ") || cl.starts_with("cz ") || cl.swap_check() {
        n_2q += 1;
    } else if !cl.starts_with("measure ") {
        n_1q += 1;
    }
}

let t_total_s = (n_1q as f64 * spec.gate_time_1q_us + n_2q as f64 * spec.gate_time_2q_us) * 1e-6;
let dephasing_err = (t_total_s / spec.t2_s).clamp(0.0, 0.40);

let t_2q_s = (n_2q as f64 * spec.gate_time_2q_us) * 1e-6;
let delta_n = spec.heating_rate_quanta_s * t_2q_s;
let motional_err = (delta_n * 0.08).clamp(0.0, 0.30);

let scatter_err = ((n_1q as f64 + 2.0 * n_2q as f64) * spec.raman_scattering_prob).clamp(0.0, 0.20);
let total_ion_err = (dephasing_err + motional_err + scatter_err).clamp(0.0005, 0.45);

let mut cum_probs = Vec::with_capacity(dim);
let mut acc = 0.0;
for &p in &probs {
    acc += p;
    cum_probs.push(acc);
}

let mut rng = rand::thread_rng();

for _ in 0..shots {
    let mut val = if rng.gen::<f64>() >= total_ion_err {
        let r: f64 = rng.gen();
        let mut chosen = 0;
        for (idx, &cp) in cum_probs.iter().enumerate() {
            if r <= cp {
                chosen = idx;
                break;
            }
        }
```

```
            chosen
        } else {
            rng.gen_range(0..dim)
        };

        for q in 0..self.num_qubits {
            let bit = (val >> q) & 1;
            if bit == 0 {
                if rng.gen::<f64>() < spec.readout_p01 {
                    val ^= 1 << q;
                }
            } else if rng.gen::<f64>() < spec.readout_p10 {
                val ^= 1 << q;
            }
        }

        let bitstring = format!("{:0width$b}", val, width = self.num_qubits);
        *results.entry(bitstring).or_insert(0) += 1;
    }

    results
}

fn apply_1q<F>(state: &mut [Complex64], n: usize, q: usize, f: F)
where
    F: Fn(Complex64, Complex64) -> (Complex64, Complex64),
{
    let dim = 1 << n;
    for i in 0..dim {
        if (i >> q) & 1 == 0 {
            let j = i | (1 << q);
            let (u, v) = f(state[i], state[j]);
            state[i] = u;
            state[j] = v;
        }
    }
}

pub fn apply_instruction(state: &mut [Complex64], n: usize, inst: &str) {
    let clean = inst.trim().trim_end_matches(';').trim();
    let dim = 1 << n;

    if clean.starts_with("h ") {
        if let Some(q) = Self::parse_qubit_idx(&clean[2..]) {
            if q < n {
                let inv = 1.0 / std::f64::consts::SQRT_2;
                Self::apply_1q(state, n, q, |u, v| ((u + v) * inv, (u - v) * inv));
            }
        }
    } else if clean.starts_with("x ") {
        if let Some(q) = Self::parse_qubit_idx(&clean[2..]) {
            if q < n {
                Self::apply_1q(state, n, q, |u, v| (v, u));
            }
        }
    } else if clean.starts_with("y ") {
        if let Some(q) = Self::parse_qubit_idx(&clean[2..]) {
            if q < n {
```

```
            let iu = Complex64::new(0.0, 1.0);
            Self::apply_1q(state, n, q, |u, v| (-iu * v, iu * u));
        }
    }
} else if clean.starts_with("z ") {
    if let Some(q) = Self::parse_qubit_idx(&clean[2..]) {
        if q < n {
            Self::apply_1q(state, n, q, |u, v| (u, -v));
        }
    }
} else if clean.starts_with("rx(") || clean.starts_with("ry(") || clean.starts_with("rz(") {
    let prefix = if clean.starts_with("rx(") {
        "rx("
    } else if clean.starts_with("ry(") {
        "ry("
    } else {
        "rz("
    };
    if let Some((angle, q)) = Self::parse_rot_gate(clean, prefix) {
        if q < n {
            let half = angle / 2.0;
            match prefix {
                "rx(" => {
                    let c = Complex64::new(half.cos(), 0.0);
                    let s_i = Complex64::new(0.0, -half.sin());
                    Self::apply_1q(state, n, q, |u, v| (u * c + v * s_i, u * s_i + v * c));
                }
                "ry(" => {
                    let c = half.cos();
                    let s = half.sin();
                    Self::apply_1q(state, n, q, |u, v| (u * c - v * s, u * s + v * c));
                }
                "rz(" => {
                    let p_neg = Complex64::new(half.cos(), -half.sin());
                    let p_pos = Complex64::new(half.cos(), half.sin());
                    Self::apply_1q(state, n, q, |u, v| (u * p_neg, v * p_pos));
                }
                _ => {}
            }
        }
    }
} else if clean.starts_with("cx ") || clean.starts_with("cnot ") {
    let rem = if clean.starts_with("cx ") {
        &clean[3..]
    } else {
        &clean[5..]
    };
    if let Some((c, t)) = Self::parse_two_qubits(rem) {
        if c < n && t < n && c != t {
            for i in 0..dim {
                if ((i >> c) & 1 == 1) && ((i >> t) & 1 == 0) {
                    state.swap(i, i | (1 << t));
                }
            }
        }
    }
} else if clean.starts_with("cz ") {
    if let Some((c, t)) = Self::parse_two_qubits(&clean[3..]) {
```

```
            if c < n && t < n && c != t {
                for i in 0..dim {
                    if ((i >> c) & 1 == 1) && ((i >> t) & 1 == 1) {
                        state[i] = -state[i];
                    }
                }
            }
        }
    } else if clean.starts_with("swap ") {
        if let Some((a, b)) = Self::parse_two_qubits(&clean[5..]) {
            if a < n && b < n && a != b {
                for i in 0..dim {
                    if ((i >> a) & 1 == 1) && ((i >> b) & 1 == 0) {
                        let j = i ^ (1 << a) ^ (1 << b);
                        state.swap(i, j);
                    }
                }
            }
        }
    }
}

fn parse_two_qubits(rem: &str) -> Option<(usize, usize)> {
    let parts: Vec<&str> = rem.split(',').collect();
    if parts.len() == 2 {
        let q1 = Self::parse_qubit_idx(parts[0])?;
        let q2 = Self::parse_qubit_idx(parts[1])?;
        Some((q1, q2))
    } else {
        None
    }
}

fn parse_rot_gate(line: &str, prefix: &str) -> Option<(f64, usize)> {
    let after = line.strip_prefix(prefix)?;
    let close = after.find(')')?;
    let angle = Self::parse_angle(&after[..close]);
    let q = Self::parse_qubit_idx(after[close + 1..].trim())?;
    Some((angle, q))
}

fn parse_angle(token: &str) -> f64 {
    let s = token.trim().to_lowercase().replace(' ', "");
    let pi = std::f64::consts::PI;
    if s.contains("pi") {
        if s == "pi" {
            pi
        } else if s == "-pi" {
            -pi
        } else if let Some(denom) = s.strip_prefix("pi/") {
            denom.parse::<f64>().map(|d| pi / d).unwrap_or(pi)
        } else if let Some(denom) = s.strip_prefix("-pi/") {
            denom.parse::<f64>().map(|d| -pi / d).unwrap_or(-pi)
        } else if s.contains("*pi/") {
            let parts: Vec<&str> = s.split("*pi/").collect();
            let num = parts.get(0).and_then(|v| v.parse::<f64>().ok()).unwrap_or(1.0);
            let den = parts.get(1).and_then(|v| v.parse::<f64>().ok()).unwrap_or(1.0);
            (num * pi) / den
```

```
            } else if s.contains("*pi") {
                s.replace("*pi", "").parse::<f64>().unwrap_or(1.0) * pi
            } else {
                pi
            }
        } else {
            s.parse::<f64>().unwrap_or(0.0)
        }
    }

    fn parse_qubit_idx(token: &str) -> Option<usize> {
        let start = token.find('[')?;
        let end = token.find(']')?;
        token[start + 1..end].trim().parse::<usize>().ok()
    }
}

trait StrExt {
    fn swap_check(&self) -> bool;
}

impl StrExt for str {
    fn swap_check(&self) -> bool {
        self.starts_with("swap ")
    }
}
```

density_matrix.rs

```
#[allow(dead_code)]
pub struct DensityMatrixSimulator {
    pub dim: usize,
    pub matrix_re: Vec<Vec<f64>>,
    pub matrix_im: Vec<Vec<f64>>,
}

#[allow(dead_code)]
impl DensityMatrixSimulator {
    pub fn new(num_qubits: usize) -> Self {
        let dim = 1 << num_qubits;
        let mut matrix_re = vec![vec![0.0; dim]; dim];
        let matrix_im = vec![vec![0.0; dim]; dim];
        matrix_re[0][0] = 1.0;

        Self { dim, matrix_re, matrix_im }
    }

    pub fn trace(&self) -> f64 {
        let mut tr = 0.0;
        for i in 0..self.dim {
            tr += self.matrix_re[i][i];
        }
        tr
    }
}
```

channels.rs

```
#[allow(dead_code)]
```

```
pub struct QuantumNoiseChannel;

#[allow(dead_code)]
impl QuantumNoiseChannel {
    pub fn apply_amplitude_damping(p1: f64, t1_us: f64, gate_time_us: f64) -> f64 {
        let gamma = 1.0 - (-gate_time_us / t1_us).exp();
        p1 * (1.0 - gamma)
    }

    pub fn apply_phase_damping(coherence: f64, t2_us: f64, gate_time_us: f64) -> f64 {
        let gamma_phi = 1.0 - (-gate_time_us / t2_us).exp();
        coherence * (1.0 - gamma_phi)
    }
}
```

circuit_grid.rs

```
use eframe::egui::Color32;
use serde::{Deserialize, Serialize};

#[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
pub enum GateType {
    H,
    X,
    Y,
    Z,
    Rx,
    Ry,
    Rz,
    Cnot,
    Cz,
    Swap,
    Measure,
}

impl GateType {
    pub fn label(&self) -> &'static str {
        match self {
            GateType::H => "H",
            GateType::X => "X",
            GateType::Y => "Y",
            GateType::Z => "Z",
            GateType::Rx => "Rx",
            GateType::Ry => "Ry",
            GateType::Rz => "Rz",
            GateType::Cnot => "CX",
            GateType::Cz => "CZ",
            GateType::Swap => "SWAP",
            GateType::Measure => "M",
        }
    }

    pub fn color(&self) -> Color32 {
        match self {
            GateType::H => Color32::from_rgb(41, 128, 185),
            GateType::X => Color32::from_rgb(192, 57, 43),
            GateType::Y => Color32::from_rgb(142, 68, 173),
            GateType::Z => Color32::from_rgb(39, 174, 96),
            GateType::Rx => Color32::from_rgb(211, 84, 0),
```

```
            GateType::Ry => Color32::from_rgb(125, 60, 152),
            GateType::Rz => Color32::from_rgb(22, 160, 133),
            GateType::Cnot => Color32::from_rgb(230, 126, 34),
            GateType::Cz => Color32::from_rgb(52, 73, 94),
            GateType::Swap => Color32::from_rgb(180, 130, 20),
            GateType::Measure => Color32::from_rgb(127, 140, 141),
        }
    }
}

#[derive(Debug, Clone, Default, Serialize, Deserialize)]
pub struct GridCell {
    pub gate: Option<GateType>,
    pub control_qubit: Option<usize>,
    pub param: Option<f64>,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
pub enum AlgorithmPreset {
    Ghz3Qubit,
    BellState,
    Grover2Qubit,
    DeutschJozsaBalanced,
    QuantumTeleportation,
}

impl AlgorithmPreset {
    pub fn name(&self) -> &'static str {
        match self {
            AlgorithmPreset::Ghz3Qubit => "3-Qubit GHZ Entanglement (|000> + |111>)",
            AlgorithmPreset::BellState => "2-Qubit Bell State Φ+ (|00> + |11>)",
            AlgorithmPreset::Grover2Qubit => "Grover's Search 2-Qubit (Marked |11>)",
            AlgorithmPreset::DeutschJozsaBalanced => "Deutsch-Jozsa (Balanced Oracle f(x)=x)",
            AlgorithmPreset::QuantumTeleportation => "Quantum Teleportation (Alice -> Bob)",
        }
    }
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct CircuitGrid {
    pub num_qubits: usize,
    pub num_steps: usize,
    pub grid: Vec<Vec<GridCell>>,
}

impl Default for CircuitGrid {
    fn default() -> Self {
        Self::from_preset(AlgorithmPreset::Ghz3Qubit)
    }
}

impl std::ops::Index<usize> for CircuitGrid {
    type Output = Vec<GridCell>;
    fn index(&self, index: usize) -> &Self::Output {
        &self.grid[index]
    }
}
```

```
impl std::ops::IndexMut<usize> for CircuitGrid {
    fn index_mut(&mut self, index: usize) -> &mut Self::Output {
        &mut self.grid[index]
    }
}

impl CircuitGrid {
    pub fn from_preset(preset: AlgorithmPreset) -> Self {
        match preset {
            AlgorithmPreset::Ghz3Qubit => {
                let num_qubits = 3;
                let num_steps = 5;
                let mut grid = vec![vec![GridCell::default(); num_steps]; num_qubits];

                grid[0][0].gate = Some(GateType::H);

                grid[1][1].gate = Some(GateType::Cnot);
                grid[1][1].control_qubit = Some(0);

                grid[2][2].gate = Some(GateType::Cnot);
                grid[2][2].control_qubit = Some(1);

                for q in 0..3 {
                    grid[q][3].gate = Some(GateType::Measure);
                }

                Self { num_qubits, num_steps, grid }
            }
            AlgorithmPreset::BellState => {
                let num_qubits = 2;
                let num_steps = 4;
                let mut grid = vec![vec![GridCell::default(); num_steps]; num_qubits];

                grid[0][0].gate = Some(GateType::H);

                grid[1][1].gate = Some(GateType::Cnot);
                grid[1][1].control_qubit = Some(0);

                grid[0][2].gate = Some(GateType::Measure);
                grid[1][2].gate = Some(GateType::Measure);

                Self { num_qubits, num_steps, grid }
            }
            AlgorithmPreset::Grover2Qubit => {
                let num_qubits = 2;
                let num_steps = 9;
                let mut grid = vec![vec![GridCell::default(); num_steps]; num_qubits];

                // 1. Equal Superposition
                grid[0][0].gate = Some(GateType::H);
                grid[1][0].gate = Some(GateType::H);

                // 2. Oracle for |11>: CZ gate marks |11> with -1 phase
                grid[1][1].gate = Some(GateType::Cz);
                grid[1][1].control_qubit = Some(0);

                // 3. Diffusion operator: H -> X -> CZ -> X -> H
                grid[0][2].gate = Some(GateType::H);
```

```
        grid[1][2].gate = Some(GateType::H);

        grid[0][3].gate = Some(GateType::X);
        grid[1][3].gate = Some(GateType::X);

        grid[1][4].gate = Some(GateType::Cz);
        grid[1][4].control_qubit = Some(0);

        grid[0][5].gate = Some(GateType::X);
        grid[1][5].gate = Some(GateType::X);

        grid[0][6].gate = Some(GateType::H);
        grid[1][6].gate = Some(GateType::H);

        // 4. Measure
        grid[0][7].gate = Some(GateType::Measure);
        grid[1][7].gate = Some(GateType::Measure);

        Self { num_qubits, num_steps, grid }
    }
    AlgorithmPreset::DeutschJozsaBalanced => {
        let num_qubits = 2;
        let num_steps = 6;
        let mut grid = vec![vec![GridCell::default(); num_steps]; num_qubits];

        // Ancilla q[1] initialized to |1>
        grid[1][0].gate = Some(GateType::X);

        // Superposition on input and ancilla (|->)
        grid[0][1].gate = Some(GateType::H);
        grid[1][1].gate = Some(GateType::H);

        // Balanced oracle f(x) = x -> CNOT(0 -> 1)
        grid[1][2].gate = Some(GateType::Cnot);
        grid[1][2].control_qubit = Some(0);

        // Interference on input
        grid[0][3].gate = Some(GateType::H);

        // Measurement: input q[0] gives |1> indicating balanced!
        grid[0][4].gate = Some(GateType::Measure);
        grid[1][4].gate = Some(GateType::Measure);

        Self { num_qubits, num_steps, grid }
    }
    AlgorithmPreset::QuantumTeleportation => {
        let num_qubits = 3;
        let num_steps = 7;
        let mut grid = vec![vec![GridCell::default(); num_steps]; num_qubits];

        // Prepare state on q[0] to teleport (|+)
        grid[0][0].gate = Some(GateType::H);

        // Entangle Bell pair between Alice q[1] and Bob q[2]
        grid[1][1].gate = Some(GateType::H);
        grid[2][2].gate = Some(GateType::Cnot);
        grid[2][2].control_qubit = Some(1);
```

```
            // Alice Bell measurement: CNOT(q0 -> q1), H(q0)
            grid[1][3].gate = Some(GateType::Cnot);
            grid[1][3].control_qubit = Some(0);

            grid[0][4].gate = Some(GateType::H);

            // Measurement
            for q in 0..3 {
                grid[q][5].gate = Some(GateType::Measure);
            }

            Self { num_qubits, num_steps, grid }
        }
    }
}

pub fn add_qubit(&mut self) {
    if self.num_qubits < 8 {
        self.num_qubits += 1;
        self.grid.push(vec![GridCell::default(); self.num_steps]);
    }
}

pub fn remove_qubit(&mut self) {
    if self.num_qubits > 1 {
        self.num_qubits -= 1;
        self.grid.pop();
        for q in 0..self.num_qubits {
            for s in 0..self.num_steps {
                if let Some(ctrl) = self.grid[q][s].control_qubit {
                    if ctrl >= self.num_qubits {
                        self.grid[q][s].control_qubit = Some(0);
                    }
                }
            }
        }
    }
}

pub fn to_qasm(&self) -> String {
    let mut qasm = format!(
        "OPENQASM 2.0;\ninclude \"qelib1.inc\";\nqreg q[{}];\ncreg c[{}];\n",
        self.num_qubits, self.num_qubits
    );

    for step in 0..self.num_steps {
        for q in 0..self.num_qubits {
            let cell = &self.grid[q][step];
            if let Some(gate) = cell.gate {
                match gate {
                    GateType::H => qasm.push_str(&format!("h q[{}];\n", q)),
                    GateType::X => qasm.push_str(&format!("x q[{}];\n", q)),
                    GateType::Y => qasm.push_str(&format!("y q[{}];\n", q)),
                    GateType::Z => qasm.push_str(&format!("z q[{}];\n", q)),
                    GateType::Rx => {
                        let th = cell.param.unwrap_or(std::f64::consts::FRAC_PI_2);
                        qasm.push_str(&format!("rx({:.4}) q[{}];\n", th, q));
                    }
```

```
                GateType::Ry => {
                    let th = cell.param.unwrap_or(std::f64::consts::FRAC_PI_2);
                    qasm.push_str(&format!("ry({:.4}) q[{}];\n", th, q));
                }
                GateType::Rz => {
                    let th = cell.param.unwrap_or(std::f64::consts::FRAC_PI_2);
                    qasm.push_str(&format!("rz({:.4}) q[{}];\n", th, q));
                }
                GateType::Cnot => {
                    let mut ctrl = cell.control_qubit.unwrap_or(if q > 0 { q - 1 } else { 1 });
                    if ctrl >= self.num_qubits || ctrl == q {
                        ctrl = if q > 0 { 0 } else { 1.min(self.num_qubits.saturating_sub(1)) };
                    }
                    if ctrl != q {
                        qasm.push_str(&format!("cx q[{}],q[{}];\n", ctrl, q));
                    }
                }
                GateType::Cz => {
                    let mut ctrl = cell.control_qubit.unwrap_or(if q > 0 { q - 1 } else { 1 });
                    if ctrl >= self.num_qubits || ctrl == q {
                        ctrl = if q > 0 { 0 } else { 1.min(self.num_qubits.saturating_sub(1)) };
                    }
                    if ctrl != q {
                        qasm.push_str(&format!("cz q[{}],q[{}];\n", ctrl, q));
                    }
                }
                GateType::Swap => {
                    let partner = cell.control_qubit.unwrap_or(if q > 0 { q - 1 } else { 1 });
                    if partner < self.num_qubits && q < partner {
                        qasm.push_str(&format!("swap q[{}],q[{}];\n", q, partner));
                    }
                }
                GateType::Measure => {
                    qasm.push_str(&format!("measure q[{}] -> c[{}];\n", q, q));
                }
            }
        }
    }
    qasm
}

pub fn from_qasm(qasm: &str) -> Result<Self, String> {
    let mut num_qubits = 3;
    for line in qasm.lines().map(str::trim) {
        if line.starts_with("qreg") {
            if let (Some(s), Some(e)) = (line.find('['), line.find(']')) {
                if let Ok(n) = line[s + 1..e].parse::<usize>() {
                    num_qubits = n.clamp(1, 8);
                    break;
                }
            }
        }
    }

    let num_steps = 10;
    let mut grid = vec![vec![GridCell::default(); num_steps]; num_qubits];
    let mut qubit_timeline = vec![0; num_qubits];
```

```
for line in qasm.lines().map(|l| l.trim().trim_end_matches(';').trim()) {
    if line.is_empty()
        || line.starts_with("//")
        || line.starts_with("OPENQASM")
        || line.starts_with("include")
        || line.starts_with("qreg")
        || line.starts_with("creg")
    {
        continue;
    }

    let single_op = if line.starts_with("h ") {
        Some((GateType::H, &line[2..]))
    } else if line.starts_with("x ") {
        Some((GateType::X, &line[2..]))
    } else if line.starts_with("y ") {
        Some((GateType::Y, &line[2..]))
    } else if line.starts_with("z ") {
        Some((GateType::Z, &line[2..]))
    } else if line.starts_with("measure ") {
        Some((GateType::Measure, &line[8..]))
    } else {
        None
    };

    if let Some((gate, target)) = single_op {
        if let Some(q) = Self::parse_qubit_idx(target) {
            if q < num_qubits {
                let st = qubit_timeline[q].min(num_steps - 1);
                grid[q][st].gate = Some(gate);
                qubit_timeline[q] = (st + 1).min(num_steps);
            }
        }
        continue;
    }

    let mut is_rot = false;
    for (prefix, gate) in [("rx(", GateType::Rx), ("ry(", GateType::Ry), ("rz(", GateType::Rz)] {
        if line.starts_with(prefix) {
            if let Some((angle, q)) = Self::parse_rot_gate(line, prefix) {
                if q < num_qubits {
                    let st = qubit_timeline[q].min(num_steps - 1);
                    grid[q][st].gate = Some(gate);
                    grid[q][st].param = Some(angle);
                    qubit_timeline[q] = (st + 1).min(num_steps);
                }
            }
            is_rot = true;
            break;
        }
    }
    if is_rot {
        continue;
    }

    let two_q = if line.starts_with("cx ") {
        Some((GateType::Cnot, &line[3..]))
```

```
        } else if line.starts_with("cnot ") {
            Some((GateType::Cnot, &line[5..]))
        } else if line.starts_with("cz ") {
            Some((GateType::Cz, &line[3..]))
        } else if line.starts_with("swap ") {
            Some((GateType::Swap, &line[5..]))
        } else {
            None
        };

        if let Some((gate, rem)) = two_q {
            let parts: Vec<&str> = rem.split(',').collect();
            if parts.len() == 2 {
                if let (Some(a), Some(b)) = (Self::parse_qubit_idx(parts[0]), Self::parse_qubit_idx(parts[1])) {
                    if a < num_qubits && b < num_qubits && a != b {
                        let st = qubit_timeline[a].max(qubit_timeline[b]).min(num_steps - 1);
                        if gate == GateType::Swap {
                            grid[a][st] = GridCell { gate: Some(GateType::Swap), control_qubit: Some(b), param: None };
                            grid[b][st] = GridCell { gate: Some(GateType::Swap), control_qubit: Some(a), param: None };
                        } else {
                            grid[b][st] = GridCell { gate: Some(gate), control_qubit: Some(a), param: None };
                        }
                        qubit_timeline[a] = (st + 1).min(num_steps);
                        qubit_timeline[b] = (st + 1).min(num_steps);
                    }
                }
            }
        }
    }

    Ok(Self {
        num_qubits,
        num_steps,
        grid,
    })
}

fn parse_rot_gate(line: &str, prefix: &str) -> Option<(f64, usize)> {
    let after = line.strip_prefix(prefix)?;
    let close = after.find(')')?;
    let angle = Self::parse_angle(&after[..close]);
    let q = Self::parse_qubit_idx(after[close + 1..].trim())?;
    Some((angle, q))
}

fn parse_angle(token: &str) -> f64 {
    let s = token.trim().to_lowercase().replace(' ', "");
    let pi = std::f64::consts::PI;
    if s.contains("pi") {
        if s == "pi" {
            pi
        } else if s == "-pi" {
            -pi
        } else if let Some(denom) = s.strip_prefix("pi/") {
            denom.parse::<f64>().map(|d| pi / d).unwrap_or(pi)
        } else if let Some(denom) = s.strip_prefix("-pi/") {
            denom.parse::<f64>().map(|d| -pi / d).unwrap_or(-pi)
        } else if s.contains("*pi/") {
```

```
            let parts: Vec<&str> = s.split("*pi/").collect();
            let num = parts.get(0).and_then(|v| v.parse::<f64>().ok()).unwrap_or(1.0);
            let den = parts.get(1).and_then(|v| v.parse::<f64>().ok()).unwrap_or(1.0);
            (num * pi) / den
        } else if s.contains("*pi") {
            s.replace("*pi", "").parse::<f64>().unwrap_or(1.0) * pi
        } else {
            pi
        }
    } else {
        s.parse::<f64>().unwrap_or(0.0)
    }
}

fn parse_qubit_idx(token: &str) -> Option<usize> {
    let start = token.find('[')?;
    let end = token.find(']')?;
    token[start + 1..end].trim().parse::<usize>().ok()
}
}
```

physical_specs.rs

```
use serde::{Deserialize, Serialize};

#[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
pub enum HardwarePlatform {
    SuperconductingTransmon,
    TrappedIon,
}

impl HardwarePlatform {
    pub fn label(&self) -> &'static str {
        match self {
            HardwarePlatform::SuperconductingTransmon => "⚛️ Superconducting Transmon",
            HardwarePlatform::TrappedIon => "⚡ Trapped Ion (Aria-1)",
        }
    }
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct PhysicalTransmonSpec {
    pub josephson_energy_ej: f64,
    pub charging_energy_ec: f64,
    pub t1_us: f64,
    pub t2_us: f64,
    pub readout_p01: f64,
    pub readout_p10: f64,
    pub flux_noise_amplitude: f64,
    pub thermal_population_nth: f64,
    pub gate_time_1q_us: f64,
    pub gate_time_2q_us: f64,
}

impl Default for PhysicalTransmonSpec {
    fn default() -> Self {
        Self {
            josephson_energy_ej: 18.0,
```

```
            charging_energy_ec: 0.28,
            t1_us: 120.0,
            t2_us: 80.0,
            readout_p01: 0.015,
            readout_p10: 0.020,
            flux_noise_amplitude: 1e-4,
            thermal_population_nth: 0.01,
            gate_time_1q_us: 0.020,
            gate_time_2q_us: 0.050,
        }
    }
}

impl PhysicalTransmonSpec {
    pub fn qubit_frequency_ghz(&self) -> f64 {
        (8.0 * self.josephson_energy_ej * self.charging_energy_ec).sqrt() - self.charging_energy_ec
    }

    pub fn ej_ec_ratio(&self) -> f64 {
        self.josephson_energy_ej / self.charging_energy_ec
    }

    pub fn anharmonicity_ghz(&self) -> f64 {
        -self.charging_energy_ec
    }
}

#[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
pub enum IonSpecies {
    Yb171, // 171Yb+ (12.64 GHz hyperfine clock qubit)
    Ca40,  // 40Ca+ (729 nm optical quadrupole qubit)
    Ba133, // 133Ba+ (visible optical/hyperfine qubit)
}

impl IonSpecies {
    pub fn name(&self) -> &'static str {
        match self {
            IonSpecies::Yb171 => "171Yb+ (Hyperfine Clock, 12.6 GHz)",
            IonSpecies::Ca40 => "40Ca+ (Optical Quadrupole, 729 nm)",
            IonSpecies::Ba133 => "133Ba+ (Barium Visible, 1.7 GHz)",
        }
    }

    pub fn natural_t1_s(&self) -> f64 {
        match self {
            IonSpecies::Yb171 => 1e6,
            IonSpecies::Ca40 => 1.16,
            IonSpecies::Ba133 => 80.0,
        }
    }
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct PhysicalIonTrapSpec {
    pub species: IonSpecies,
    pub name: String,
    pub t1_s: f64,
    pub t2_s: f64,
```

```
    pub secular_freq_axial_mhz: f64,
    pub secular_freq_radial_mhz: f64,
    pub heating_rate_quanta_s: f64, // \dot{\bar{n}} motional heating
    pub gate_time_1q_us: f64,
    pub gate_time_2q_us: f64,       // Mølmer-Sørensen gate
    pub raman_scattering_prob: f64,
    pub readout_p01: f64,
    pub readout_p10: f64,
    pub all_to_all_connectivity: bool,
}

impl Default for PhysicalIonTrapSpec {
    fn default() -> Self {
        Self::ionq_aria1()
    }
}

impl PhysicalIonTrapSpec {
    pub fn ionq_aria1() -> Self {
        Self {
            species: IonSpecies::Yb171,
            name: "IonQ Aria-1 (Yb-171)".to_string(),
            t1_s: 1000.0,
            t2_s: 1.2,
            secular_freq_axial_mhz: 0.35,
            secular_freq_radial_mhz: 3.10,
            heating_rate_quanta_s: 8.0,
            gate_time_1q_us: 10.0,
            gate_time_2q_us: 180.0,
            raman_scattering_prob: 0.0003,
            readout_p01: 0.003,
            readout_p10: 0.005,
            all_to_all_connectivity: true,
        }
    }

    pub fn optical_ca40() -> Self {
        Self {
            species: IonSpecies::Ca40,
            name: "Optical Ca-40 Trap".to_string(),
            t1_s: 1.16,
            t2_s: 0.25,
            secular_freq_axial_mhz: 0.80,
            secular_freq_radial_mhz: 2.50,
            heating_rate_quanta_s: 25.0,
            gate_time_1q_us: 5.0,
            gate_time_2q_us: 100.0,
            raman_scattering_prob: 0.0010,
            readout_p01: 0.006,
            readout_p10: 0.008,
            all_to_all_connectivity: true,
        }
    }

    pub fn high_noise_surface() -> Self {
        Self {
            species: IonSpecies::Yb171,
            name: "High-Noise Surface Trap".to_string(),
```

```
            t1_s: 50.0,
            t2_s: 0.08,
            secular_freq_axial_mhz: 0.25,
            secular_freq_radial_mhz: 1.80,
            heating_rate_quanta_s: 150.0,
            gate_time_1q_us: 15.0,
            gate_time_2q_us: 250.0,
            raman_scattering_prob: 0.0025,
            readout_p01: 0.020,
            readout_p10: 0.035,
            all_to_all_connectivity: true,
        }
    }
}
```

postprocessing.rs

```
use serde::{Deserialize, Serialize};
use std::collections::HashMap;

/// Pre-configured Quantum Error Correction (QEC) Code Presets
#[derive(Debug, Clone, PartialEq, Serialize, Deserialize)]
pub enum QECCodePreset {
    Repetition3QubitBitFlip, // 3 physical -> 1 logical qubit
    ErrorDetecting4Qubit,    // 4 physical -> 2 logical qubits (arXiv:2412.04791v2)
    Perfect5Qubit,           // 5 physical -> 1 logical qubit
    Steane7Qubit,            // 7 physical -> 1 logical qubit CSS code
    Custom,                  // User-defined physical (N) -> logical (K) mapping
}

impl QECCodePreset {
    pub fn name(&self) -> &'static str {
        match self {
            QECCodePreset::Repetition3QubitBitFlip => "3-Qubit Bit-Flip Repetition Code",
            QECCodePreset::ErrorDetecting4Qubit => "4-Qubit Error-Detecting Code (Deutsch-Jozsa)",
            QECCodePreset::Perfect5Qubit => "5-Qubit Perfect Code",
            QECCodePreset::Steane7Qubit => "7-Qubit Steane CSS Code",
            QECCodePreset::Custom => "Custom N-Qubit Code",
        }
    }

    pub fn physical_qubits(&self) -> usize {
        match self {
            QECCodePreset::Repetition3QubitBitFlip => 3,
            QECCodePreset::ErrorDetecting4Qubit => 4,
            QECCodePreset::Perfect5Qubit => 5,
            QECCodePreset::Steane7Qubit => 7,
            QECCodePreset::Custom => 3,
        }
    }

    #[allow(dead_code)]
    pub fn logical_qubits(&self) -> usize {
        match self {
            QECCodePreset::Repetition3QubitBitFlip => 1,
            QECCodePreset::ErrorDetecting4Qubit => 2,
            QECCodePreset::Perfect5Qubit => 1,
            QECCodePreset::Steane7Qubit => 1,
```

```
            QECCodePreset::Custom => 1,
        }
    }
}

#[derive(Debug, Clone, PartialEq, Serialize, Deserialize)]
pub enum DecodeMode {
    ActiveSyndromeCorrection,
    PostSelectionDiscarding,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct CustomSyndromeRule {
    pub syndrome_bits: String,
    pub flip_qubit_idx: usize,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct QECCodeConfig {
    pub preset: QECCodePreset,
    pub decode_mode: DecodeMode,
    pub enable_auto_majority_vote: bool,
    pub custom_rules: Vec<CustomSyndromeRule>,
    pub custom_script_code: String,
}

impl Default for QECCodeConfig {
    fn default() -> Self {
        let default_rules = vec![
            CustomSyndromeRule { syndrome_bits: "10".to_string(), flip_qubit_idx: 0 },
            CustomSyndromeRule { syndrome_bits: "11".to_string(), flip_qubit_idx: 1 },
            CustomSyndromeRule { syndrome_bits: "01".to_string(), flip_qubit_idx: 2 },
        ];

        Self {
            preset: QECCodePreset::Repetition3QubitBitFlip,
            decode_mode: DecodeMode::ActiveSyndromeCorrection,
            enable_auto_majority_vote: true,
            custom_rules: default_rules,
            custom_script_code: String::new(),
        }
    }
}

#[derive(Debug, Clone)]
pub struct QECPipelineResult {
    pub corrected_physical_counts: HashMap<String, usize>,
    pub logical_counts: HashMap<String, usize>,
    #[allow(dead_code)]
    pub syndrome_histogram: HashMap<String, usize>,
    pub total_input_shots: usize,
    pub retained_shots: usize,
    pub discarded_shots: usize,
    pub retention_rate: f64,
    #[allow(dead_code)]
    pub logical_error_rate: f64,
}
```

```
pub struct Postprocessor;

impl Postprocessor {
    pub fn mitigate_readout_errors_1q(
        raw_counts: &HashMap<String, usize>,
        total_shots: usize,
        p01: f64,
        p10: f64,
    ) -> HashMap<String, f64> {
        if total_shots == 0 {
            return HashMap::new();
        }
        let n0 = *raw_counts.get("0").unwrap_or(&0) as f64 / total_shots as f64;
        let n1 = *raw_counts.get("1").unwrap_or(&0) as f64 / total_shots as f64;

        let det = 1.0 - p10 - p01;
        if det.abs() < 1e-6 {
            let mut res = HashMap::new();
            res.insert("0".to_string(), n0);
            res.insert("1".to_string(), n1);
            return res;
        }

        let p0_ideal = ((1.0 - p01) * n0 - p01 * n1) / det;
        let p1_ideal = (-p10 * n0 + (1.0 - p10) * n1) / det;

        let p0_clamped = p0_ideal.clamp(0.0, 1.0);
        let p1_clamped = p1_ideal.clamp(0.0, 1.0);
        let sum = (p0_clamped + p1_clamped).max(1e-9);

        let mut mitigated = HashMap::new();
        mitigated.insert("0".to_string(), p0_clamped / sum);
        mitigated.insert("1".to_string(), p1_clamped / sum);
        mitigated
    }

    pub fn process_qec_pipeline(
        raw_counts: &HashMap<String, usize>,
        config: &QECCodeConfig,
    ) -> QECPipelineResult {
        let mut corrected_physical: HashMap<String, usize> = HashMap::new();
        let mut logical_counts: HashMap<String, usize> = HashMap::new();
        let mut syndrome_histogram: HashMap<String, usize> = HashMap::new();

        let mut total_input_shots = 0;
        let mut retained_shots = 0;
        let mut discarded_shots = 0;
        let mut uncorrected_logical_errors = 0;

        for (bitstring, &count) in raw_counts {
            total_input_shots += count;

            match config.preset {
                QECCodePreset::Repetition3QubitBitFlip => {
                    if bitstring.len() != 3 {
                        *corrected_physical.entry(bitstring.clone()).or_insert(0) += count;
                        retained_shots += count;
                        continue;
```

```
    }

    let bits: Vec<u8> = bitstring.chars().map(|c| if c == '1' { 1 } else { 0 }).collect();
    let (s1, s2) = (bits[0] ^ bits[1], bits[1] ^ bits[2]);
    let syndrome_str = format!("{}{}", s1, s2);
    *syndrome_histogram.entry(syndrome_str.clone()).or_insert(0) += count;

    if config.decode_mode == DecodeMode::PostSelectionDiscarding {
        if syndrome_str != "00" {
            discarded_shots += count;
            continue;
        }
    }

    let mut corrected_bits = bits.clone();
    if config.decode_mode == DecodeMode::ActiveSyndromeCorrection {
        if config.enable_auto_majority_vote {
            match syndrome_str.as_str() {
                "10" => corrected_bits[0] ^= 1,
                "11" => corrected_bits[1] ^= 1,
                "01" => corrected_bits[2] ^= 1,
                _ => {}
            }
        } else {
            for rule in &config.custom_rules {
                if rule.syndrome_bits == syndrome_str && rule.flip_qubit_idx < corrected_bits.len() {
                    corrected_bits[rule.flip_qubit_idx] ^= 1;
                    break;
                }
            }
        }
    }

    retained_shots += count;
    let corr_str: String = corrected_bits.iter().map(|b| if *b == 1 { '1' } else { '0' }).collect();
    *corrected_physical.entry(corr_str.clone()).or_insert(0) += count;

    let logical_bit = if corr_str == "111" { "1" } else { "0" };
    *logical_counts.entry(logical_bit.to_string()).or_insert(0) += count;

    if corr_str != "000" && corr_str != "111" {
        uncorrected_logical_errors += count;
    }
}

QECCodePreset::ErrorDetecting4Qubit => {
    if bitstring.len() != 4 {
        *corrected_physical.entry(bitstring.clone()).or_insert(0) += count;
        retained_shots += count;
        continue;
    }

    let bits: Vec<u8> = bitstring.chars().map(|c| if c == '1' { 1 } else { 0 }).collect();
    let parity = (bits[0] + bits[1] + bits[2] + bits[3]) % 2;
    let syndrome_str = format!("P{}", parity);
    *syndrome_histogram.entry(syndrome_str.clone()).or_insert(0) += count;

    if config.decode_mode == DecodeMode::PostSelectionDiscarding {
```

```
                if parity != 0 {
                    discarded_shots += count;
                    continue;
                }
            }

            let mut corrected_bits = bits.clone();
            if config.decode_mode == DecodeMode::ActiveSyndromeCorrection {
                if !config.enable_auto_majority_vote {
                    for rule in &config.custom_rules {
                        if rule.syndrome_bits == syndrome_str && rule.flip_qubit_idx < corrected_bits.len() {
                            corrected_bits[rule.flip_qubit_idx] ^= 1;
                            break;
                        }
                    }
                }
            }

            retained_shots += count;
            let corr_str: String = corrected_bits.iter().map(|b| if *b == 1 { '1' } else { '0' }).collect();
            *corrected_physical.entry(corr_str.clone()).or_insert(0) += count;

            let logical_state = match corr_str.as_str() {
                "0000" | "1111" => "00",
                "1100" | "0011" => "01",
                "1010" | "0101" => "10",
                "0110" | "1001" => "11",
                _ => "Unmapped",
            };
            *logical_counts.entry(logical_state.to_string()).or_insert(0) += count;
        }

        _ => {
            let mut corrected_bits: Vec<u8> = bitstring.chars().map(|c| if c == '1' { 1 } else { 0 }).collect();
            let syndrome_str = if corrected_bits.len() >= 2 {
                format!("{}{}", corrected_bits[0] ^ corrected_bits[1], corrected_bits[1] ^
corrected_bits.last().copied().unwrap_or(0))
            } else {
                "0".to_string()
            };
            *syndrome_histogram.entry(syndrome_str.clone()).or_insert(0) += count;

            if config.decode_mode == DecodeMode::ActiveSyndromeCorrection && !config.enable_auto_majority_vote {
                for rule in &config.custom_rules {
                    if rule.syndrome_bits == syndrome_str && rule.flip_qubit_idx < corrected_bits.len() {
                        corrected_bits[rule.flip_qubit_idx] ^= 1;
                        break;
                    }
                }
            }

            retained_shots += count;
            let corr_str: String = corrected_bits.iter().map(|b| if *b == 1 { '1' } else { '0' }).collect();
            *corrected_physical.entry(corr_str.clone()).or_insert(0) += count;
            let logical_str = corr_str.chars().next().unwrap_or('0').to_string();
            *logical_counts.entry(logical_str).or_insert(0) += count;
        }
    }
```

```
        }

        let retention_rate = if total_input_shots > 0 {
            retained_shots as f64 / total_input_shots as f64
        } else {
            0.0
        };

        let logical_error_rate = if retained_shots > 0 {
            uncorrected_logical_errors as f64 / retained_shots as f64
        } else {
            0.0
        };

        QECPipelineResult {
            corrected_physical_counts: corrected_physical,
            logical_counts,
            syndrome_histogram,
            total_input_shots,
            retained_shots,
            discarded_shots,
            retention_rate,
            logical_error_rate,
        }
    }

    pub fn extrapolate_zero_noise(scale_factors: &[f64], values: &[f64]) -> f64 {
        if scale_factors.len() < 2 || scale_factors.len() != values.len() {
            return values.first().copied().unwrap_or(0.0);
        }

        let n = scale_factors.len() as f64;
        let sum_x: f64 = scale_factors.iter().sum();
        let sum_y: f64 = values.iter().sum();
        let sum_xy: f64 = scale_factors.iter().zip(values.iter()).map(|(x, y)| x * y).sum();
        let sum_x2: f64 = scale_factors.iter().map(|x| x * x).sum();

        let denom = n * sum_x2 - sum_x * sum_x;
        if denom.abs() < 1e-9 {
            return sum_y / n;
        }

        let slope = (n * sum_xy - sum_x * sum_y) / denom;
        (sum_y - slope * sum_x) / n
    }

    pub fn generate_python_postprocessing_script(config: &QECCodeConfig) -> String {
        format!(
            r#"# Auto-Generated In-Engine Postprocessing Script
# Active Preset: {} ({})

def in_engine_postprocess(raw_shots):
    retained_shots = {{}}
    discarded_count = 0

    for bitstring, count in raw_shots.items():
        if len(bitstring) == {}:
            retained_shots[bitstring] = count
```

```
        else:
            discarded_count += count

    return retained_shots, discarded_count

raw_shots = {{"0000": 1950, "1111": 1920, "1000": 110, "0110": 60}}
retained, discarded = in_engine_postprocess(raw_shots)
print("Retained Shots :", retained)
print("Discarded Shots:", discarded)
"#,
            config.preset.name(),
            if config.decode_mode == DecodeMode::ActiveSyndromeCorrection { "Active Correction" } else { "Post-
Selection" },
            config.preset.physical_qubits()
        )
    }
}
```

app.rs

```
use eframe::egui::{self, Color32, Pos2, Stroke, Vec2};
use std::collections::HashMap;

use crate::circuit_grid::{AlgorithmPreset, CircuitGrid, GateType, GridCell};
use crate::emitters::{
    EmitterErrorEstimator, OpticsHbtHom, PurcellCavitySpec, SiliconDefect,
};
use crate::physical_specs::{
    HardwarePlatform, IonSpecies, PhysicalIonTrapSpec, PhysicalTransmonSpec,
};
use crate::postprocessing::{
    CustomSyndromeRule, DecodeMode, Postprocessor, QECCodeConfig, QECCodePreset,
};
use crate::qasm::QasmProgram;

#[derive(PartialEq, Debug, Clone, Copy)]
pub enum AppMode {
    TransmonQubits,
    QuantumEmitters,
    PostprocessingQec,
}

#[derive(PartialEq, Debug, Clone, Copy)]
pub enum PostprocessProtocol {
    ReadoutMitigation,
    QecStabilizer,
    ZeroNoiseExtrapolation,
}

pub struct TransmonApp {
    pub mode: AppMode,

    // --- Hardware Platform & Physics Specs ---
    pub hardware_platform: HardwarePlatform,
    pub spec: PhysicalTransmonSpec,
    pub ion_spec: PhysicalIonTrapSpec,

    // --- Circuit & Simulation State ---
```

```
    pub grid: CircuitGrid,
    pub selected_preset: Option<AlgorithmPreset>,
    pub selected_gate: Option<GateType>,
    pub selected_cnot_control: usize,
    pub selected_angle: f64,
    pub qasm_text: String,
    pub shots: usize,
    pub simulation_results: Option<HashMap<String, usize>>,
    pub transmon_error_msg: Option<String>,

    // --- Quantum Emitter State ---
    pub selected_defect: SiliconDefect,
    pub cavity_q: f64,
    pub mode_volume_lambda3: f64,
    pub coupling_g_ghz: f64,
    pub cavity_loss_kappa_ghz: f64,
    pub detuning_ghz: f64,
    pub spin_bath_noise_mhz: f64,

    // --- Postprocessing & QEC State ---
    pub postprocessing_protocol: PostprocessProtocol,
    pub qec_config: QECCodeConfig,
    pub custom_python_script: String,
    pub zne_scale_factors: Vec<f64>,
    pub zne_measured_vals: Vec<f64>,
    pub readout_p01_mitigation: f64,
    pub readout_p10_mitigation: f64,
    pub new_syndrome_input: String,
    pub new_qubit_idx_input: usize,
}

impl Default for TransmonApp {
    fn default() -> Self {
        let grid = CircuitGrid::default();
        let qasm_text = grid.to_qasm();
        let qec_config = QECCodeConfig::default();
        let custom_python_script = Postprocessor::generate_python_postprocessing_script(&qec_config);

        let mut app = Self {
            mode: AppMode::TransmonQubits,

            // Hardware & Specs
            hardware_platform: HardwarePlatform::SuperconductingTransmon,
            spec: PhysicalTransmonSpec::default(),
            ion_spec: PhysicalIonTrapSpec::ionq_aria1(),

            // Circuit Defaults
            grid,
            selected_preset: Some(AlgorithmPreset::Ghz3Qubit),
            selected_gate: Some(GateType::H),
            selected_cnot_control: 0,
            selected_angle: std::f64::consts::FRAC_PI_2,
            qasm_text,
            shots: 1024,
            simulation_results: None,
            transmon_error_msg: None,

            // Emitter Defaults
```

```
            selected_defect: SiliconDefect::GCenter,
            cavity_q: 10000.0,
            mode_volume_lambda3: 0.80,
            coupling_g_ghz: 0.25,
            cavity_loss_kappa_ghz: 0.10,
            detuning_ghz: 0.0,
            spin_bath_noise_mhz: 0.20,

            // Postprocessing Defaults
            postprocessing_protocol: PostprocessProtocol::QecStabilizer,
            qec_config,
            custom_python_script,
            zne_scale_factors: vec![1.0, 1.5, 2.0, 3.0],
            zne_measured_vals: vec![0.85, 0.76, 0.68, 0.52],
            readout_p01_mitigation: 0.03,
            readout_p10_mitigation: 0.04,
            new_syndrome_input: "10".to_string(),
            new_qubit_idx_input: 0,
        };
        app.run_transmon_simulation();
        app
    }
}

impl TransmonApp {
    fn sync_from_grid(&mut self) {
        self.qasm_text = self.grid.to_qasm();
        self.run_transmon_simulation();
    }

    fn run_transmon_simulation(&mut self) {
        self.transmon_error_msg = None;
        match QasmProgram::from_str(&self.qasm_text) {
            Ok(program) => {
                let results = match self.hardware_platform {
                    HardwarePlatform::SuperconductingTransmon => {
                        program.run_noisy_simulation(&self.spec, self.shots)
                    }
                    HardwarePlatform::TrappedIon => {
                        program.run_noisy_simulation_ion(&self.ion_spec, self.shots)
                    }
                };
                self.simulation_results = Some(results);
            }
            Err(err) => {
                self.transmon_error_msg = Some(err);
                self.simulation_results = None;
            }
        }
    }

    // ========================================================================
    // WORKSPACE 1: QUANTUM HARDWARE & CIRCUIT SIMULATION
    // ========================================================================
    fn render_transmon_mode(&mut self, ctx: &egui::Context) {
        egui::SidePanel::left("hardware_controls")
            .default_width(310.0)
            .min_width(250.0)
```

```
            .resizable(true)
            .show(ctx, |ui| {
                ui.heading("Quantum Hardware Console");
                ui.add_space(4.0);

                // Hardware Platform Selector: Transmon vs. Trapped Ion
                ui.horizontal(|ui| {
                    let transmon_btn = ui.selectable_label(
                        self.hardware_platform == HardwarePlatform::SuperconductingTransmon,
                        "⚛ Transmon",
                    );
                    if transmon_btn.clicked() {
                        self.hardware_platform = HardwarePlatform::SuperconductingTransmon;
                        self.run_transmon_simulation();
                    }

                    let ion_btn = ui.selectable_label(
                        self.hardware_platform == HardwarePlatform::TrappedIon,
                        "⚡ Trapped Ion",
                    );
                    if ion_btn.clicked() {
                        self.hardware_platform = HardwarePlatform::TrappedIon;
                        self.run_transmon_simulation();
                    }
                });
                ui.separator();

                egui::ScrollArea::vertical().show(ui, |ui| {
                    match self.hardware_platform {
                        HardwarePlatform::SuperconductingTransmon => {
                            ui.group(|ui| {
                                ui.label(egui::RichText::new("Transmon Physical
Parameters").strong().color(Color32::LIGHT_BLUE));
                                ui.add(egui::Slider::new(&mut self.spec.josephson_energy_ej, 5.0..=35.0).text("E_J (GHz)"));
                                ui.add(egui::Slider::new(&mut self.spec.charging_energy_ec, 0.10..=0.50).text("E_C (GHz)"));

                                let freq = self.spec.qubit_frequency_ghz();
                                let ratio = self.spec.ej_ec_ratio();
                                ui.label(format!("Transmon Freq f_01 : {:.2} GHz", freq));
                                ui.label(format!("E_J / E_C Ratio   : {:.1}", ratio));
                                ui.label(format!("Anharmonicity α   : {:.2} GHz", self.spec.anharmonicity_ghz()));
                                if ratio < 20.0 {
                                    ui.colored_label(Color32::KHAKI, "Warning: Low E_J/E_C (<20) increases charge noise.");
                                }
                            });

                            ui.add_space(8.0);
                            ui.group(|ui| {
                                ui.label(egui::RichText::new("Decoherence & Relaxation Times").strong());
                                ui.add(egui::Slider::new(&mut self.spec.t1_us, 1.0..=300.0).text("T_1 Decay (us)"));
                                ui.add(egui::Slider::new(&mut self.spec.t2_us, 1.0..=300.0).text("T_2 Dephasing (us)"));
                            });

                            ui.add_space(8.0);
                            ui.group(|ui| {
                                ui.label(egui::RichText::new("Readout Confusion Matrix").strong());
                                ui.add(egui::Slider::new(&mut self.spec.readout_p01, 0.0..=0.15).text("P(1|0) False Positive"));
                                ui.add(egui::Slider::new(&mut self.spec.readout_p10, 0.0..=0.15).text("P(0|1) False Negative"));
```

```
                });

                ui.add_space(8.0);
                ui.group(|ui| {
                    ui.label(egui::RichText::new("Transmon Presets").strong());
                    ui.horizontal(|ui| {
                        if ui.button("IBM Eagle").clicked() {
                            self.spec = PhysicalTransmonSpec {
                                josephson_energy_ej: 18.0,
                                charging_energy_ec: 0.28,
                                t1_us: 120.0,
                                t2_us: 80.0,
                                readout_p01: 0.012,
                                readout_p10: 0.018,
                                flux_noise_amplitude: 1e-4,
                                thermal_population_nth: 0.01,
                                gate_time_1q_us: 0.02,
                                gate_time_2q_us: 0.05,
                            };
                            self.run_transmon_simulation();
                        }
                        if ui.button("High Noise").clicked() {
                            self.spec = PhysicalTransmonSpec {
                                josephson_energy_ej: 12.0,
                                charging_energy_ec: 0.35,
                                t1_us: 15.0,
                                t2_us: 10.0,
                                readout_p01: 0.06,
                                readout_p10: 0.08,
                                flux_noise_amplitude: 5e-4,
                                thermal_population_nth: 0.04,
                                gate_time_1q_us: 0.02,
                                gate_time_2q_us: 0.05,
                            };
                            self.run_transmon_simulation();
                        }
                    });
                });
            }
            HardwarePlatform::TrappedIon => {
                ui.group(|ui| {
                    ui.label(egui::RichText::new("Trapped-Ion Architecture").strong().color(Color32::from_rgb(241, 196,
15)));
                    ui.label(format!("Natural Atomic T_1: {:.1} s", self.ion_spec.species.natural_t1_s()));
                    egui::ComboBox::from_label("Species")
                        .selected_text(self.ion_spec.species.name())
                        .show_ui(ui, |ui| {
                            ui.selectable_value(&mut self.ion_spec.species, IonSpecies::Yb171, "171Yb+ (Clock, 12.6
GHz)");
                            ui.selectable_value(&mut self.ion_spec.species, IonSpecies::Ca40, "40Ca+ (Optical, 729 nm)");
                            ui.selectable_value(&mut self.ion_spec.species, IonSpecies::Ba133, "133Ba+ (Visible, 1.7 GHz)");
                        });

                    ui.colored_label(Color32::from_rgb(46, 204, 113), "⚡ Native All-to-All Coulomb Crystal Bus
Active");
                });

                ui.add_space(8.0);
```

```
                    ui.group(|ui| {
                        ui.label(egui::RichText::new("Secular Trapping & Phonon Noise").strong());
                        ui.add(egui::Slider::new(&mut self.ion_spec.secular_freq_radial_mhz, 0.5..=5.0).text("Radial Freq
(MHz)"));
                        ui.add(egui::Slider::new(&mut self.ion_spec.heating_rate_quanta_s,
0.1..=200.0).logarithmic(true).text("Motional Heating dn/dt (quanta/s)"));
                        ui.add(egui::Slider::new(&mut self.ion_spec.t2_s, 0.01..=10.0).logarithmic(true).text("T_2 Coherence
(s)"));
                    });

                    ui.add_space(8.0);
                    ui.group(|ui| {
                        ui.label(egui::RichText::new("Raman Gate Durations & Optical Errors").strong());
                        ui.add(egui::Slider::new(&mut self.ion_spec.gate_time_2q_us, 50.0..=400.0).text("MS Gate 2Q (us)"));
                        ui.add(egui::Slider::new(&mut self.ion_spec.raman_scattering_prob,
0.0001..=0.005).logarithmic(true).text("Raman Scatter Err"));
                        ui.add(egui::Slider::new(&mut self.ion_spec.readout_p01, 0.001..=0.05).text("Fluorescence P(1|0)"));
                        ui.add(egui::Slider::new(&mut self.ion_spec.readout_p10, 0.001..=0.05).text("Fluorescence P(0|1)"));
                    });

                    ui.add_space(8.0);
                    ui.group(|ui| {
                        ui.label(egui::RichText::new("Ion Trap Presets").strong());
                        ui.horizontal(|ui| {
                            if ui.button("IonQ Aria-1").clicked() {
                                self.ion_spec = PhysicalIonTrapSpec::ionq_aria1();
                                self.run_transmon_simulation();
                            }
                            if ui.button("Optical Ca-40").clicked() {
                                self.ion_spec = PhysicalIonTrapSpec::optical_ca40();
                                self.run_transmon_simulation();
                            }
                            if ui.button("High Noise").clicked() {
                                self.ion_spec = PhysicalIonTrapSpec::high_noise_surface();
                                self.run_transmon_simulation();
                            }
                        });
                    });
                }
            }

            ui.add_space(8.0);
            ui.group(|ui| {
                ui.label(egui::RichText::new("Execution Shots").strong());
                ui.add(egui::Slider::new(&mut self.shots, 128..=8192).text("Shots"));
            });
        });
    });

    egui::CentralPanel::default().show(ctx, |ui| {
        // Preset Algorithm Toolbar (Phase 1 P0)
        ui.horizontal(|ui| {
            ui.heading("Circuit Composer");
            ui.separator();
            ui.label(egui::RichText::new("📚 Preset Algorithms:").strong().color(Color32::from_rgb(241, 196, 15)));

            let current_name = self.selected_preset.map(|p| p.name()).unwrap_or("Custom Circuit");
            egui::ComboBox::from_id_source("preset_algo_dropdown")
```

```
        .selected_text(current_name)
        .show_ui(ui, |ui| {
            let presets = [
                AlgorithmPreset::Ghz3Qubit,
                AlgorithmPreset::BellState,
                AlgorithmPreset::Grover2Qubit,
                AlgorithmPreset::DeutschJozsaBalanced,
                AlgorithmPreset::QuantumTeleportation,
            ];
            for p in presets {
                if ui.selectable_label(self.selected_preset == Some(p), p.name()).clicked() {
                    self.selected_preset = Some(p);
                    self.grid = CircuitGrid::from_preset(p);
                    self.sync_from_grid();
                }
            }
        });
});
ui.separator();

// Gate Palette Toolbar
ui.horizontal(|ui| {
    ui.label(egui::RichText::new("Gate Palette:").strong());
    let gates = [
        GateType::H,
        GateType::X,
        GateType::Y,
        GateType::Z,
        GateType::Rx,
        GateType::Ry,
        GateType::Rz,
        GateType::Cnot,
        GateType::Cz,
        GateType::Swap,
        GateType::Measure,
    ];
    for g in gates {
        let is_sel = self.selected_gate == Some(g);
        let stroke = if is_sel {
            Stroke::new(2.5_f32, Color32::YELLOW)
        } else {
            Stroke::NONE
        };
        let text_color = Color32::WHITE;
        let btn_text = if is_sel {
            format!("[*] {}", g.label())
        } else {
            g.label().to_string()
        };

        let btn = egui::Button::new(
            egui::RichText::new(btn_text).color(text_color).strong(),
        )
        .fill(g.color())
        .stroke(stroke);

        if ui.add(btn).clicked() {
            self.selected_gate = Some(g);
```

```
            }
        }

        if matches!(self.selected_gate, Some(GateType::Rx | GateType::Ry | GateType::Rz)) {
            ui.separator();
            ui.label(egui::RichText::new("Angle θ:").strong().color(Color32::LIGHT_BLUE));
            ui.add(egui::Slider::new(&mut self.selected_angle, -std::f64::consts::PI..=std::f64::consts::PI).text("rad"));
            if ui.button("π/4").clicked() {
                self.selected_angle = std::f64::consts::FRAC_PI_4;
            }
            if ui.button("π/2").clicked() {
                self.selected_angle = std::f64::consts::FRAC_PI_2;
            }
            if ui.button("π").clicked() {
                self.selected_angle = std::f64::consts::PI;
            }
            if ui.button("-π/2").clicked() {
                self.selected_angle = -std::f64::consts::FRAC_PI_2;
            }
        } else if matches!(self.selected_gate, Some(GateType::Cnot | GateType::Cz | GateType::Swap)) {
            ui.separator();
            let lbl = if self.selected_gate == Some(GateType::Swap) {
                "Partner:"
            } else {
                "Control:"
            };
            ui.label(egui::RichText::new(lbl).strong().color(Color32::LIGHT_BLUE));
            egui::ComboBox::from_id_source("multi_q_combo")
                .selected_text(format!("q[{}]", self.selected_cnot_control.min(self.grid.num_qubits.saturating_sub(1))))
                .show_ui(ui, |ui| {
                    for q_idx in 0..self.grid.num_qubits {
                        ui.selectable_value(&mut self.selected_cnot_control, q_idx, format!("q[{}]", q_idx));
                    }
                });
        }

        ui.separator();
        if ui.button("[+] Qubit").clicked() {
            self.grid.add_qubit();
            self.selected_preset = None;
            self.sync_from_grid();
        }
        if ui.button("[-] Qubit").clicked() {
            self.grid.remove_qubit();
            self.selected_preset = None;
            self.sync_from_grid();
        }
        if ui.button("[Run Grid]").clicked() {
            self.sync_from_grid();
        }
    });

    ui.add_space(8.0);

    let mut grid_changed = false;
    ui.group(|ui| {
        ui.horizontal(|ui| {
            ui.label(egui::RichText::new("Circuit Wire Grid").strong());
```

```
    ui.label("(Click on wire to place selected gate. Multi-qubit gates use control/partner above)");
});
for q in 0..self.grid.num_qubits {
    ui.horizontal(|ui| {
        ui.label(egui::RichText::new(format!("q[{}] --", q)).monospace().strong());
        for s in 0..self.grid.num_steps {
            let cell = self.grid[q][s].clone();

            let is_control_for = (0..self.grid.num_qubits).find(|&other_q| {
                other_q != q
                    && (self.grid.grid[other_q][s].gate == Some(GateType::Cnot)
                        || self.grid.grid[other_q][s].gate == Some(GateType::Cz))
                    && self.grid.grid[other_q][s].control_qubit == Some(q)
            });

            let (text, fill) = if let Some(g) = cell.gate {
                match g {
                    GateType::Rx | GateType::Ry | GateType::Rz => {
                        let th = cell.param.unwrap_or(0.0);
                        (format!("{}({:.2})", g.label(), th), g.color())
                    }
                    GateType::Cnot => {
                        let ctrl = cell.control_qubit.unwrap_or(0);
                        (format!("(+)c{}", ctrl), g.color())
                    }
                    GateType::Cz => {
                        let ctrl = cell.control_qubit.unwrap_or(0);
                        (format!("(+)z{}", ctrl), g.color())
                    }
                    GateType::Swap => {
                        let p = cell.control_qubit.unwrap_or(0);
                        (format!("x q{}", p), g.color())
                    }
                    _ => (g.label().to_string(), g.color()),
                }
            } else if let Some(target_q) = is_control_for {
                (format!("• ->q{}", target_q), Color32::from_rgb(41, 128, 185))
            } else {
                (" -- ".to_string(), Color32::from_gray(50))
            };

            if ui
                .add(
                    egui::Button::new(
                        egui::RichText::new(text).color(Color32::WHITE),
                    )
                    .fill(fill),
                )
                .clicked()
            {
                self.selected_preset = None;
                if let Some(g) = self.selected_gate {
                    match g {
                        GateType::Rx | GateType::Ry | GateType::Rz => {
                            if cell.gate == Some(g) && cell.param == Some(self.selected_angle) {
                                self.grid[q][s].gate = None;
                                self.grid[q][s].param = None;
                            } else {
```

```
            self.grid[q][s].gate = Some(g);
            self.grid[q][s].param = Some(self.selected_angle);
        }
        grid_changed = true;
    }
    GateType::Cnot | GateType::Cz => {
        let ctrl = if self.selected_cnot_control != q {
            self.selected_cnot_control
        } else if q > 0 {
            q - 1
        } else {
            1.min(self.grid.num_qubits.saturating_sub(1))
        };

        if cell.gate == Some(g) {
            self.grid[q][s].gate = None;
            self.grid[q][s].control_qubit = None;
        } else {
            self.grid[q][s].gate = Some(g);
            self.grid[q][s].control_qubit = Some(ctrl);
        }
        grid_changed = true;
    }
    GateType::Swap => {
        let partner = if self.selected_cnot_control != q {
            self.selected_cnot_control
        } else {
            (q + 1) % self.grid.num_qubits
        };
        if cell.gate == Some(GateType::Swap) {
            self.grid[q][s] = GridCell::default();
            self.grid[partner][s] = GridCell::default();
        } else {
            self.grid[q][s] = GridCell {
                gate: Some(GateType::Swap),
                control_qubit: Some(partner),
                param: None,
            };
            self.grid[partner][s] = GridCell {
                gate: Some(GateType::Swap),
                control_qubit: Some(q),
                param: None,
            };
        }
        grid_changed = true;
    }
    _ => {
        if cell.gate == Some(g) {
            self.grid[q][s].gate = None;
            self.grid[q][s].control_qubit = None;
            self.grid[q][s].param = None;
        } else {
            self.grid[q][s].gate = Some(g);
            self.grid[q][s].control_qubit = None;
            self.grid[q][s].param = None;
        }
        grid_changed = true;
    }
```

```
                    }
                }
            }
        }
    });
}

if grid_changed {
    self.sync_from_grid();
}

ui.add_space(10.0);

// QASM Editor & Direct Input Mode
ui.group(|ui| {
    ui.horizontal(|ui| {
        ui.label(egui::RichText::new("OpenQASM 2.0 Direct Editor").strong());
        ui.separator();
        if ui.button("▶️ Run from QASM Text").clicked() {
            self.run_transmon_simulation();
        }
        if ui.button("🔄 Sync QASM -> Circuit Grid").clicked() {
            match CircuitGrid::from_qasm(&self.qasm_text) {
                Ok(new_grid) => {
                    self.grid = new_grid;
                    self.selected_preset = None;
                    self.run_transmon_simulation();
                }
                Err(e) => {
                    self.transmon_error_msg = Some(e);
                }
            }
        }
        if ui.button("🔄 Sync Grid -> QASM").clicked() {
            self.sync_from_grid();
        }
    });

    ui.add(
        egui::TextEdit::multiline(&mut self.qasm_text)
            .font(egui::TextStyle::Monospace)
            .desired_rows(7)
            .desired_width(f32::INFINITY),
    );
});
});

// Right Panel: Visual Graphical Histogram (Phase 1 P0)
egui::SidePanel::right("transmon_results")
    .default_width(300.0)
    .min_width(240.0)
    .resizable(true)
    .show(ctx, |ui| {
        ui.heading("Hardware Shot Histogram");
        ui.separator();

        if let Some(err) = &self.transmon_error_msg {
```

```
                ui.colored_label(Color32::LIGHT_RED, format!("Error: {}", err));
            } else if let Some(counts) = &self.simulation_results {
                ui.horizontal(|ui| {
                    ui.label(egui::RichText::new(format!("Platform: {}", self.hardware_platform.label())).strong());
                });
                ui.label(format!("Total Shots Executed: {}", self.shots));
                ui.add_space(8.0);

                let mut sorted_keys: Vec<_> = counts.keys().collect();
                sorted_keys.sort();

                egui::ScrollArea::vertical().show(ui, |ui| {
                    ui.group(|ui| {
                        ui.label(egui::RichText::new("Measured State Distribution").strong());
                        ui.add_space(4.0);

                        for k in sorted_keys {
                            let cnt = counts[k];
                            let pct = (cnt as f64 / self.shots as f64) * 100.0;
                            let frac = (cnt as f32 / self.shots as f32).clamp(0.0, 1.0);

                            ui.horizontal(|ui| {
                                ui.label(egui::RichText::new(format!("|{}>", k)).monospace().strong());

                                // Graphical color bar
                                let bar_color = if frac > 0.40 {
                                    Color32::from_rgb(46, 204, 113) // Emerald Green for dominant peaks
                                } else if frac > 0.15 {
                                    Color32::from_rgb(52, 152, 219) // Dodger Blue for superpositions
                                } else {
                                    Color32::from_rgb(127, 140, 141) // Slate Gray for noise floor
                                };

                                let (rect, _response) = ui.allocate_exact_size(Vec2::new(100.0, 14.0), egui::Sense::hover());
                                let painter = ui.painter_at(rect);

                                // Background track
                                painter.rect_filled(rect, 2.0, Color32::from_rgb(30, 36, 48));

                                // Filled bar
                                let mut fill_rect = rect;
                                fill_rect.set_width((rect.width() * frac).max(1.0));
                                painter.rect_filled(fill_rect, 2.0, bar_color);

                                ui.label(egui::RichText::new(format!("{:4} ({:4.1}%)", cnt, pct)).monospace());
                            });
                        }
                    });
                });
            }
        });
}

// ============================================================================
// WORKSPACE 2: SILICON QUANTUM EMITTERS
// ============================================================================
fn render_emitters_mode(&mut self, ctx: &egui::Context) {
    egui::SidePanel::left("emitter_controls")
```

```
    .default_width(320.0)
    .min_width(250.0)
    .resizable(true)
    .show(ctx, |ui| {
        ui.heading("Silicon Quantum Emitters");
        ui.separator();

        egui::ScrollArea::vertical().show(ui, |ui| {
            ui.group(|ui| {
                ui.label(egui::RichText::new("1. Silicon Color Center Defect").strong());
                egui::ComboBox::from_label("Defect Registry")
                    .selected_text(self.selected_defect.name())
                    .show_ui(ui, |ui| {
                        ui.selectable_value(&mut self.selected_defect, SiliconDefect::GCenter, "G-Center (1278 nm)");
                        ui.selectable_value(&mut self.selected_defect, SiliconDefect::TCenter, "T-Center (1326 nm)");
                        ui.selectable_value(&mut self.selected_defect, SiliconDefect::CiCenter, "C_i-Center (1280 nm)");
                        ui.selectable_value(&mut self.selected_defect, SiliconDefect::Al1Center, "Al_1-Center (1300 nm)");
                        ui.selectable_value(&mut self.selected_defect, SiliconDefect::WCenter, "W-Center (1218 nm)");
                        ui.selectable_value(&mut self.selected_defect, SiliconDefect::Er3Plus, "Er^3+ Ion (1536 nm)");
                    });

                ui.label(format!("Emission Wavelength lambda : {:.1} nm", self.selected_defect.wavelength_nm()));
                ui.label(format!("Zero-Phonon Line (ZPL)   : {:.0}%", self.selected_defect.zpl_fraction() * 100.0));
                ui.label(format!("Lifetime T_1             : {:.1} ns", self.selected_defect.lifetime_ns()));
                ui.label(format!("Spin Coherence T_2       : {:.1} us", self.selected_defect.spin_t2_us()));
            });

            ui.add_space(8.0);
            ui.group(|ui| {
                ui.label(egui::RichText::new("2. Nanophotonic Purcell Cavity").strong());
                ui.add(egui::Slider::new(&mut self.cavity_q, 100.0..=100000.0).logarithmic(true).text("Cavity Q-Factor"));
                ui.add(egui::Slider::new(&mut self.mode_volume_lambda3, 0.1..=5.0).text("Mode Vol V (lambda/n)^3"));
                ui.add(egui::Slider::new(&mut self.coupling_g_ghz, 0.01..=2.0).text("Coupling g (GHz)"));
                ui.add(egui::Slider::new(&mut self.cavity_loss_kappa_ghz, 0.01..=5.0).text("Cavity Loss kappa (GHz)"));
                ui.add(egui::Slider::new(&mut self.detuning_ghz, -2.0..=2.0).text("Detuning Delta (GHz)"));
            });

            ui.add_space(8.0);
            ui.group(|ui| {
                ui.label(egui::RichText::new("3. Optical Dephasing & Noise").strong());
                ui.add(egui::Slider::new(&mut self.spin_bath_noise_mhz, 0.01..=10.0).text("Spin Bath Noise (MHz)"));
            });
        });
    });

let cavity_spec = PurcellCavitySpec {
    defect: self.selected_defect,
    cavity_q: self.cavity_q,
    mode_volume_lambda3: self.mode_volume_lambda3,
    coupling_g_ghz: self.coupling_g_ghz,
    cavity_loss_kappa_ghz: self.cavity_loss_kappa_ghz,
    detuning_ghz: self.detuning_ghz,
};

let f_purcell = EmitterErrorEstimator::calculate_purcell_factor(&cavity_spec);
let rabi_points = EmitterErrorEstimator::compute_rabi_dynamics(&cavity_spec, 100);
let v_hom = OpticsHbtHom::calculate_hom_visibility(&self.selected_defect, self.spin_bath_noise_mhz);
let hahn_points = OpticsHbtHom::compute_hahn_echo_decay(&self.selected_defect, self.spin_bath_noise_mhz, 100);
```

```
egui::CentralPanel::default().show(ctx, |ui| {
    ui.heading("Quantum Emitter Dynamics & Single-Photon Optics");
    ui.separator();

    egui::ScrollArea::vertical().show(ui, |ui| {
        // --- PLOT 1: Vacuum Rabi Dynamics ---
        ui.group(|ui| {
            ui.label(egui::RichText::new("Vacuum Rabi Dynamics P_e(t)").strong());
            ui.label("Time-evolution of excited state population under non-Hermitian Hamiltonian H_eff:");
            ui.add_space(4.0);

            let (rect, _response) = ui.allocate_exact_size(Vec2::new(ui.available_width(), 160.0), egui::Sense::hover());
            let painter = ui.painter_at(rect);

            painter.rect_filled(rect, 4.0, Color32::from_rgb(20, 24, 32));
            painter.rect_stroke(rect, 4.0, Stroke::new(1.0_f32, Color32::from_rgb(50, 60, 80)));

            for ratio in [0.25_f32, 0.50_f32, 0.75_f32] {
                let y = rect.bottom() - ratio * rect.height();
                painter.line_segment(
                    [Pos2::new(rect.left(), y), Pos2::new(rect.right(), y)],
                    Stroke::new(0.5_f32, Color32::from_gray(40)),
                );
            }

            if rabi_points.len() > 1 {
                let max_t = rabi_points.last().map(|p| p.0).unwrap_or(10.0);
                let points_to_draw: Vec<Pos2> = rabi_points
                    .iter()
                    .map(|(t, pe)| {
                        let x = rect.left() + (t / max_t) as f32 * rect.width();
                        let y = rect.bottom() - (*pe as f32 * 0.90_f32 + 0.05_f32) * rect.height();
                        Pos2::new(x, y)
                    })
                    .collect();

                for window in points_to_draw.windows(2) {
                    painter.line_segment([window[0], window[1]], Stroke::new(2.0_f32, Color32::LIGHT_BLUE));
                }
            }

            ui.horizontal(|ui| {
                ui.label(egui::RichText::new("Time: 0.0 ns").small().monospace());
                ui.with_layout(egui::Layout::right_to_left(egui::Align::Center), |ui| {
                    ui.label(egui::RichText::new("10.0 ns").small().monospace());
                });
            });
        });

        ui.add_space(12.0);

        // --- PLOT 2: Hahn Echo Spin Coherence Decay ---
        ui.group(|ui| {
            ui.label(egui::RichText::new("Hahn Echo Spin Coherence Decay S(t)").strong());
            ui.label(format!("HOM Photon Indistinguishability Visibility V_HOM: {:.1}%", v_hom * 100.0));
            ui.add_space(4.0);
```

```
            let (rect, _response) = ui.allocate_exact_size(Vec2::new(ui.available_width(), 160.0), egui::Sense::hover());
            let painter = ui.painter_at(rect);

            painter.rect_filled(rect, 4.0, Color32::from_rgb(20, 24, 32));
            painter.rect_stroke(rect, 4.0, Stroke::new(1.0_f32, Color32::from_rgb(50, 60, 80)));

            for ratio in [0.25_f32, 0.50_f32, 0.75_f32] {
                let y = rect.bottom() - ratio * rect.height();
                painter.line_segment(
                    [Pos2::new(rect.left(), y), Pos2::new(rect.right(), y)],
                    Stroke::new(0.5_f32, Color32::from_gray(40)),
                );
            }

            if hahn_points.len() > 1 {
                let max_t = hahn_points.last().map(|p| p.0).unwrap_or(10.0);
                let points_to_draw: Vec<Pos2> = hahn_points
                    .iter()
                    .map(|(t, sig)| {
                        let x = rect.left() + (t / max_t) as f32 * rect.width();
                        let y = rect.bottom() - (*sig as f32 * 0.90_f32 + 0.05_f32) * rect.height();
                        Pos2::new(x, y)
                    })
                    .collect();

                for window in points_to_draw.windows(2) {
                    painter.line_segment([window[0], window[1]], Stroke::new(2.0_f32, Color32::GREEN));
                }
            }

            let max_t_us = hahn_points.last().map(|p| p.0).unwrap_or(0.0);
            ui.horizontal(|ui| {
                ui.label(egui::RichText::new("Time: 0.0 us").small().monospace());
                ui.with_layout(egui::Layout::right_to_left(egui::Align::Center), |ui| {
                    ui.label(egui::RichText::new(format!("{:.1} us", max_t_us)).small().monospace());
                });
            });
        });
    });

    egui::SidePanel::right("emitter_metrics")
        .default_width(260.0)
        .min_width(200.0)
        .resizable(true)
        .show(ctx, |ui| {
            ui.heading("Emitter & Cavity Metrics");
            ui.separator();

            ui.group(|ui| {
                ui.label(egui::RichText::new("Cavity Purcell Enhancement").strong());
                ui.label(format!("Purcell Factor F_P : {:.1}x", f_purcell));
                let enhanced_lifetime = self.selected_defect.lifetime_ns() / (1.0 + f_purcell *
self.selected_defect.zpl_fraction());
                ui.label(format!("Enhanced T_1      : {:.2} ns", enhanced_lifetime));
            });

            ui.add_space(8.0);
```

```
            ui.group(|ui| {
                ui.label(egui::RichText::new("Regime Classification").strong());
                if self.coupling_g_ghz > self.cavity_loss_kappa_ghz / 4.0 {
                    ui.colored_label(Color32::GREEN, "Strong Coupling Regime (Vacuum Rabi)");
                } else {
                    ui.colored_label(Color32::LIGHT_BLUE, "Weak Coupling Regime (Purcell Decay)");
                }
            });
        });
}

// ============================================================================
// WORKSPACE 3: POSTPROCESSING & QUANTUM ERROR CORRECTION (QEC)
// ============================================================================
fn render_postprocessing_mode(&mut self, ctx: &egui::Context) {
    let raw_counts = if let Some(res) = &self.simulation_results {
        res.clone()
    } else {
        let mut mock = HashMap::new();
        mock.insert("0000".to_string(), 1950);
        mock.insert("1111".to_string(), 1920);
        mock.insert("1000".to_string(), 110);
        mock.insert("0110".to_string(), 60);
        mock.insert("1001".to_string(), 56);
        mock
    };

    egui::SidePanel::left("postprocess_controls")
        .default_width(320.0)
        .min_width(260.0)
        .resizable(true)
        .show(ctx, |ui| {
            ui.heading("QEC & Postprocessing Engine");
            ui.separator();

            egui::ScrollArea::vertical().show(ui, |ui| {
                ui.group(|ui| {
                    ui.label(egui::RichText::new("1. Select Engine Protocol").strong());
                    ui.selectable_value(
                        &mut self.postprocessing_protocol,
                        PostprocessProtocol::QecStabilizer,
                        "Stabilizer QEC Pipeline",
                    );
                    ui.selectable_value(
                        &mut self.postprocessing_protocol,
                        PostprocessProtocol::ReadoutMitigation,
                        "Readout Confusion Mitigation",
                    );
                    ui.selectable_value(
                        &mut self.postprocessing_protocol,
                        PostprocessProtocol::ZeroNoiseExtrapolation,
                        "Zero-Noise Extrapolation (ZNE)",
                    );
                });

                ui.add_space(8.0);
```

```
match self.postprocessing_protocol {
    PostprocessProtocol::QecStabilizer => {
        ui.group(|ui| {
            ui.label(egui::RichText::new("2. QEC Code Preset").strong());
            egui::ComboBox::from_label("Code Selection")
                .selected_text(self.qec_config.preset.name())
                .show_ui(ui, |ui| {
                    ui.selectable_value(
                        &mut self.qec_config.preset,
                        QECCodePreset::Repetition3QubitBitFlip,
                        "3-Qubit Bit-Flip Repetition Code",
                    );
                    ui.selectable_value(
                        &mut self.qec_config.preset,
                        QECCodePreset::ErrorDetecting4Qubit,
                        "4-Qubit Error-Detecting Code",
                    );
                    ui.selectable_value(
                        &mut self.qec_config.preset,
                        QECCodePreset::Perfect5Qubit,
                        "5-Qubit Perfect Code",
                    );
                    ui.selectable_value(
                        &mut self.qec_config.preset,
                        QECCodePreset::Steane7Qubit,
                        "7-Qubit Steane Code",
                    );
                });

            ui.add_space(6.0);
            ui.label(egui::RichText::new("3. Pipeline Strategy").strong());
            ui.selectable_value(
                &mut self.qec_config.decode_mode,
                DecodeMode::ActiveSyndromeCorrection,
                "Active Syndrome Bit-Flipping",
            );
            ui.selectable_value(
                &mut self.qec_config.decode_mode,
                DecodeMode::PostSelectionDiscarding,
                "Post-Selection Discarding",
            );
        });

        if self.qec_config.decode_mode == DecodeMode::ActiveSyndromeCorrection {
            ui.add_space(6.0);
            ui.group(|ui| {
                ui.label(egui::RichText::new("Syndrome Recovery Rules").strong());
                ui.checkbox(&mut self.qec_config.enable_auto_majority_vote, "Use Standard Codebook Lookup");

                if !self.qec_config.enable_auto_majority_vote {
                    ui.separator();
                    ui.label(egui::RichText::new("Active Custom Rules:").small().strong());
                    let mut to_remove = None;
                    for (idx, rule) in self.qec_config.custom_rules.iter().enumerate() {
                        ui.horizontal(|ui| {
                            ui.label(format!("Synd [{}] -> Flip Q[{}]", rule.syndrome_bits, rule.flip_qubit_idx));
                            if ui.button("❌").clicked() {
                                to_remove = Some(idx);
```

```
                        }
                    });
                }
                if let Some(rem_idx) = to_remove {
                    self.qec_config.custom_rules.remove(rem_idx);
                }

                ui.add_space(6.0);
                ui.label(egui::RichText::new("Add Custom Rule:").small().strong());
                ui.horizontal(|ui| {
                    ui.label("Synd:");
                    ui.add(egui::TextEdit::singleline(&mut self.new_syndrome_input).desired_width(45.0));
                    ui.label("Flip Q:");
                    let max_q = self.qec_config.preset.physical_qubits().saturating_sub(1);
                    ui.add(egui::DragValue::new(&mut self.new_qubit_idx_input).clamp_range(0..=max_q));
                    if ui.button("➕ Add").clicked() {
                        let synd = self.new_syndrome_input.trim().to_string();
                        if !synd.is_empty() {
                            self.qec_config.custom_rules.push(CustomSyndromeRule {
                                syndrome_bits: synd,
                                flip_qubit_idx: self.new_qubit_idx_input,
                            });
                        }
                    }
                });

                ui.add_space(4.0);
                if ui.button("↻ Reset Defaults").clicked() {
                    self.qec_config.custom_rules = vec![
                        CustomSyndromeRule { syndrome_bits: "10".to_string(), flip_qubit_idx: 0 },
                        CustomSyndromeRule { syndrome_bits: "11".to_string(), flip_qubit_idx: 1 },
                        CustomSyndromeRule { syndrome_bits: "01".to_string(), flip_qubit_idx: 2 },
                    ];
                }
            }
        });
    }
}
PostprocessProtocol::ReadoutMitigation => {
    ui.group(|ui| {
        ui.label(egui::RichText::new("Readout Confusion Parameters").strong());
        ui.add(egui::Slider::new(&mut self.readout_p01_mitigation, 0.0..=0.20).text("P(1|0) False Pos"));
        ui.add(egui::Slider::new(&mut self.readout_p10_mitigation, 0.0..=0.20).text("P(0|1) False Neg"));
    });
}
PostprocessProtocol::ZeroNoiseExtrapolation => {
    ui.group(|ui| {
        ui.label(egui::RichText::new("ZNE Noise Scaling Factors").strong());
        ui.label("Noise scale factors lambda: [1.0, 1.5, 2.0, 3.0]");
    });
}
}
});
});

egui::CentralPanel::default().show(ctx, |ui| {
ui.heading("Postprocessing Results & In-Engine Console");
ui.separator();
```

```
        match self.postprocessing_protocol {
            PostprocessProtocol::QecStabilizer => {
                let qec_result = Postprocessor::process_qec_pipeline(&raw_counts, &self.qec_config);

                egui::ScrollArea::vertical().show(ui, |ui| {
                    ui.group(|ui| {
                        ui.label(egui::RichText::new("ℹ️ What is Stabilizer QEC & Syndrome Decoding?").strong().color(Color32::LIGHT_BLUE));
                        ui.label(
                            "Quantum Error Correction protects quantum data by encoding logical qubits into entangled physical registers:\n\
                             • Syndrome Measurement: Parity-check stabilizer operators detect bit-flips without measuring or destroying superposition data.\n\
                             • Active Syndrome Bit-Flipping: Non-zero syndrome bitstrings map to targeted Pauli-X operations on corrupted physical qubits.\n\
                             • Post-Selection: Discards runs with detected parity violations (e.g. odd parity), boosting fidelity at the cost of retention."
                        );
                    });

                    ui.add_space(8.0);
                    ui.group(|ui| {
                        ui.label(egui::RichText::new("Physical State Histogram Comparison").strong());
                        let mut sorted_keys: Vec<_> = raw_counts.keys().cloned().collect();
                        for k in qec_result.corrected_physical_counts.keys() {
                            if !sorted_keys.contains(k) {
                                sorted_keys.push(k.clone());
                            }
                        }
                        sorted_keys.sort();

                        for key in &sorted_keys {
                            let raw_cnt = *raw_counts.get(key).unwrap_or(&0);
                            let corr_cnt = *qec_result.corrected_physical_counts.get(key).unwrap_or(&0);
                            ui.horizontal(|ui| {
                                ui.label(egui::RichText::new(format!("|{}>", key)).monospace().strong());
                                ui.label(format!("Raw: {:4}  |  Processed: {:4}", raw_cnt, corr_cnt));
                            });
                        }
                    });

                    ui.add_space(12.0);
                    ui.group(|ui| {
                        ui.horizontal(|ui| {
                            ui.label(egui::RichText::new("Option B: In-Engine Script Console").strong());
                            if ui.button("Refresh Template").clicked() {
                                self.custom_python_script =
                                    Postprocessor::generate_python_postprocessing_script(&self.qec_config);
                            }
                        });

                        ui.add(
                            egui::TextEdit::multiline(&mut self.custom_python_script)
                                .font(egui::TextStyle::Monospace)
                                .desired_rows(8)
                                .desired_width(f32::INFINITY),
                        );
```

```
                });
            });
        }
        PostprocessProtocol::ReadoutMitigation => {
            let total_shots: usize = raw_counts.values().sum();
            let mitigated = Postprocessor::mitigate_readout_errors_1q(
                &raw_counts,
                total_shots,
                self.readout_p01_mitigation,
                self.readout_p10_mitigation,
            );

            egui::ScrollArea::vertical().show(ui, |ui| {
                ui.group(|ui| {
                    ui.label(egui::RichText::new("ℹ️ What is Readout Confusion
Mitigation?").strong().color(Color32::LIGHT_BLUE));
                    ui.label(
                        "Measurement detectors on physical hardware experience classification errors:\n\
                         • False Positive P(1|0): Qubit in |0> misclassified as |1> due to thermal excitation or resonator ring-up.
\n\
                         • False Negative P(0|1): Qubit in |1> decays to |0> during measurement readout integration.\n\n\
                         Matrix Inversion Unfolding:\n\
                         [P_0_mit, P_1_mit]^T = M^(-1) * [P_0_raw, P_1_raw]^T,\n\
                         where M = [[1 - P(1|0), P(0|1)], [P(1|0), 1 - P(0|1)]].\n\
                         Inverting M removes detector distortion and recovers the ideal pre-readout quantum probabilities."
                    );
                });

                ui.add_space(8.0);
                ui.group(|ui| {
                    ui.label(egui::RichText::new("Readout Confusion Matrix Inversion").strong());
                    ui.label("Raw Measured Probabilities vs. Inverted Unfolded Distribution:");
                    ui.add_space(8.0);

                    for state in ["0", "1"] {
                        let raw_cnt = *raw_counts.get(state).unwrap_or(&0);
                        let raw_p = if total_shots > 0 {
                            raw_cnt as f64 / total_shots as f64
                        } else {
                            0.0
                        };
                        let mit_p = *mitigated.get(state).unwrap_or(&0.0);

                        ui.group(|ui| {
                            ui.label(format!("State |{}> : Raw P = {:.1}%  -->  Mitigated P = {:.1}%", state, raw_p * 100.0,
mit_p * 100.0));
                        });
                    }
                });
            });
        }
        PostprocessProtocol::ZeroNoiseExtrapolation => {
            let zne_val = Postprocessor::extrapolate_zero_noise(&self.zne_scale_factors, &self.zne_measured_vals);

            egui::ScrollArea::vertical().show(ui, |ui| {
                ui.group(|ui| {
                    ui.label(egui::RichText::new("ℹ️ What is Zero-Noise Extrapolation
(ZNE)?").strong().color(Color32::LIGHT_BLUE));
```

```
                    ui.label(
                        "Zero-Noise Extrapolation (ZNE) is a NISQ error mitigation technique requiring no extra physical qubits:\n\
                         1. Noise Amplification (λ): Artificially amplify gate noise by factors λ (e.g. 1.0x, 1.5x, 2.0x, 3.0x)\n\
                            via pulse stretching or identity insertion U -> U (U† U)^n.\n\
                         2. Expectation Sampling: Measure observable expectation values <O>(λ) at each amplified noise level.\n\
                         3. Zero-Noise Extrapolation: Fit a curve across measured points and extrapolate to the zero-noise limit λ -> 0.\n\n\
                         The intercept at λ = 0 recovers the error-free expectation value without physical ancillae."
                    );
                });

                ui.add_space(8.0);
                ui.group(|ui| {
                    ui.label(egui::RichText::new("Zero-Noise Extrapolation (ZNE) Plot").strong());
                    ui.label(format!("Extrapolated Zero-Noise Expectation Value E(lambda=0): {:.3}", zne_val));
                    ui.add_space(8.0);

                    let (rect, _response) = ui.allocate_exact_size(Vec2::new(ui.available_width(), 160.0), egui::Sense::hover());
                    let painter = ui.painter_at(rect);

                    painter.rect_filled(rect, 4.0, Color32::from_rgb(20, 24, 32));
                    painter.rect_stroke(rect, 4.0, Stroke::new(1.0_f32, Color32::from_rgb(50, 60, 80)));

                    for idx in 0..self.zne_scale_factors.len() {
                        let sf = self.zne_scale_factors[idx];
                        let val = self.zne_measured_vals[idx];
                        let x = rect.left() + (sf as f32 / 3.5_f32) * rect.width();
                        let y = rect.bottom() - (val as f32 * 0.80_f32 + 0.10_f32) * rect.height();
                        painter.circle_filled(Pos2::new(x, y), 4.0_f32, Color32::YELLOW);
                    }

                    // Extrapolated point at lambda=0
                    let x0 = rect.left();
                    let y0 = rect.bottom() - (zne_val as f32 * 0.80_f32 + 0.10_f32) * rect.height();
                    painter.circle_filled(Pos2::new(x0, y0), 6.0_f32, Color32::GREEN);
                });
            });
        }
    }
});

egui::SidePanel::right("postprocess_output")
    .default_width(270.0)
    .min_width(200.0)
    .resizable(true)
    .show(ctx, |ui| {
        ui.heading("QEC Metrics & Retention");
        ui.separator();

        if self.postprocessing_protocol == PostprocessProtocol::QecStabilizer {
            let qec_result = Postprocessor::process_qec_pipeline(&raw_counts, &self.qec_config);
            egui::ScrollArea::vertical().show(ui, |ui| {
                ui.group(|ui| {
                    ui.label(egui::RichText::new("Execution Metrics").strong());
                    ui.label(format!("Total Input Shots : {}", qec_result.total_input_shots));
```

```
                    ui.label(format!("Retained Shots    : {}", qec_result.retained_shots));
                    ui.label(format!("Discarded Shots   : {}", qec_result.discarded_shots));
                    ui.label(format!("Retention Rate    : {:.2}%", qec_result.retention_rate * 100.0));
                });

                ui.add_space(8.0);
                ui.group(|ui| {
                    ui.label(egui::RichText::new("Logical State Distribution").strong());
                    let mut sorted_logical: Vec<_> = qec_result.logical_counts.keys().collect();
                    sorted_logical.sort();

                    for l_state in sorted_logical {
                        let cnt = qec_result.logical_counts[l_state];
                        let pct = if qec_result.retained_shots > 0 {
                            (cnt as f64 / qec_result.retained_shots as f64) * 100.0
                        } else {
                            0.0
                        };
                        ui.label(format!("|{}>_L : {} shots ({:.1}%)", l_state, cnt, pct));
                    }
                });
            });
        }
    });
  }
}

impl eframe::App for TransmonApp {
  fn update(&mut self, ctx: &egui::Context, _frame: &mut eframe::Frame) {
    egui::TopBottomPanel::top("top_navigation").show(ctx, |ui| {
      ui.add_space(4.0);
      ui.horizontal(|ui| {
        ui.heading("Qty Quantum Hardware Console");
        ui.add_space(20.0);
        ui.label(egui::RichText::new("Workspace:").strong());

        ui.selectable_value(
          &mut self.mode,
          AppMode::TransmonQubits,
          "⚛️ Hardware & Circuits",
        );
        ui.selectable_value(
          &mut self.mode,
          AppMode::QuantumEmitters,
          "🔬 Silicon Emitters",
        );
        ui.selectable_value(
          &mut self.mode,
          AppMode::PostprocessingQec,
          "🛡️ Postprocessing & QEC",
        );
      });
      ui.add_space(4.0);
    });

    match self.mode {
      AppMode::TransmonQubits => self.render_transmon_mode(ctx),
      AppMode::QuantumEmitters => self.render_emitters_mode(ctx),
```

```
            AppMode::PostprocessingQec => self.render_postprocessing_mode(ctx),
        }
    }
}
```

emitters/defects.rs

```
use serde::{Deserialize, Serialize};

#[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
pub enum SiliconDefect {
    GCenter,   // 1278 nm
    TCenter,   // 1326 nm
    CiCenter,  // 1280 nm
    Al1Center, // 1300 nm
    WCenter,   // 1218 nm
    Er3Plus,   // 1536 nm
}

impl SiliconDefect {
    pub fn name(&self) -> &'static str {
        match self {
            SiliconDefect::GCenter => "G-Center (Carbon Pair + Interstitial)",
            SiliconDefect::TCenter => "T-Center (Carbon-Hydrogen Complex)",
            SiliconDefect::CiCenter => "C_i-Center (Interstitial Carbon)",
            SiliconDefect::Al1Center => "Al_1-Center (Aluminum-Related Defect)",
            SiliconDefect::WCenter => "W-Center (Tri-Interstitial Cluster)",
            SiliconDefect::Er3Plus => "Er^3+ Erbium Ion in Silicon",
        }
    }

    pub fn wavelength_nm(&self) -> f64 {
        match self {
            SiliconDefect::GCenter => 1278.4,
            SiliconDefect::TCenter => 1326.0,
            SiliconDefect::CiCenter => 1280.0,
            SiliconDefect::Al1Center => 1300.2,
            SiliconDefect::WCenter => 1218.0,
            SiliconDefect::Er3Plus => 1536.0,
        }
    }

    pub fn zpl_fraction(&self) -> f64 {
        match self {
            SiliconDefect::GCenter => 0.15,
            SiliconDefect::TCenter => 0.23,
            SiliconDefect::CiCenter => 0.10,
            SiliconDefect::Al1Center => 0.30,
            SiliconDefect::WCenter => 0.08,
            SiliconDefect::Er3Plus => 0.90,
        }
    }

    pub fn lifetime_ns(&self) -> f64 {
        match self {
            SiliconDefect::GCenter => 35.0,
            SiliconDefect::TCenter => 940.0,
            SiliconDefect::CiCenter => 25.0,
```

```
            SiliconDefect::Al1Center => 45.0,
            SiliconDefect::WCenter => 20.0,
            SiliconDefect::Er3Plus => 1_000_000.0,
        }
    }

    pub fn spin_t2_us(&self) -> f64 {
        match self {
            SiliconDefect::GCenter => 2.0,
            SiliconDefect::TCenter => 2100.0,
            SiliconDefect::CiCenter => 1.5,
            SiliconDefect::Al1Center => 10.0,
            SiliconDefect::WCenter => 0.5,
            SiliconDefect::Er3Plus => 10000.0,
        }
    }
}
```

emitters/optics_hbt_hom.rs

```
use crate::emitters::defects::SiliconDefect;

pub struct OpticsHbtHom;

impl OpticsHbtHom {
    pub fn calculate_hom_visibility(defect: &SiliconDefect, spin_bath_noise_mhz: f64) -> f64 {
        let t1_ns = defect.lifetime_ns();
        let t2_us = defect.spin_t2_us();
        let t2_ns = t2_us * 1000.0;

        // Pure dephasing rate from spin bath noise
        let gamma_pure = spin_bath_noise_mhz / 1000.0; // GHz equivalent
        let t2_star_ns = 1.0 / ((1.0 / t2_ns) + gamma_pure);

        // HOM visibility V_HOM = T2 / (2 * T1)
        (t2_star_ns / (2.0 * t1_ns)).clamp(0.0, 1.0)
    }

    pub fn compute_hahn_echo_decay(
        defect: &SiliconDefect,
        spin_bath_noise_mhz: f64,
        num_points: usize,
    ) -> Vec<(f64, f64)> {
        let t2_us = defect.spin_t2_us();
        let gamma_bath = spin_bath_noise_mhz / 10.0; // Bath noise dephasing factor
        let t2_eff = t2_us / (1.0 + gamma_bath);

        let max_t = (t2_eff * 3.0).max(1.0);
        let mut points = Vec::with_capacity(num_points);

        for i in 0..num_points {
            let t = (i as f64 / (num_points - 1) as f64) * max_t;
            // Hahn echo Gaussian decay profile
            let decay = (-(t / t2_eff).powi(2)).exp();
            points.push((t, decay));
        }

        points
```

```
    }
}
```

emitters/mod.rs

```
pub mod defects;
pub mod purcell_cavity;
pub mod optics_hbt_hom;

pub use defects::SiliconDefect;
pub use purcell_cavity::{EmitterErrorEstimator, PurcellCavitySpec};
pub use optics_hbt_hom::OpticsHbtHom;
```

emitters/purcell_cavity.rs

```
use crate::emitters::defects::SiliconDefect;

#[derive(Debug, Clone)]
pub struct PurcellCavitySpec {
    pub defect: SiliconDefect,
    pub cavity_q: f64,
    pub mode_volume_lambda3: f64,
    pub coupling_g_ghz: f64,
    pub cavity_loss_kappa_ghz: f64,
    pub detuning_ghz: f64,
}

pub struct EmitterErrorEstimator;

impl EmitterErrorEstimator {
    pub fn calculate_purcell_factor(spec: &PurcellCavitySpec) -> f64 {
        let n = 3.45; // Silicon refractive index
        let f_max = (3.0 / (4.0 * std::f64::consts::PI * std::f64::consts::PI)) * (spec.cavity_q / spec.mode_volume_lambda3) *
(1.0 / (n * n * n));
        let lorentzian = 1.0 / (1.0 + 4.0 * (spec.detuning_ghz / spec.cavity_loss_kappa_ghz).powi(2));
        f_max * lorentzian * spec.defect.zpl_fraction()
    }

    pub fn compute_rabi_dynamics(spec: &PurcellCavitySpec, num_points: usize) -> Vec<(f64, f64)> {
        let mut points = Vec::with_capacity(num_points);
        let max_time_ns = 10.0;
        let g = spec.coupling_g_ghz * 2.0 * std::f64::consts::PI;
        let kappa = spec.cavity_loss_kappa_ghz * 2.0 * std::f64::consts::PI;
        let delta = spec.detuning_ghz * 2.0 * std::f64::consts::PI;

        let omega_rabi = (g * g + (delta / 2.0).powi(2)).sqrt();

        for i in 0..num_points {
            let t = (i as f64 / (num_points - 1) as f64) * max_time_ns;
            let env = (-kappa * t / 4.0).exp();
            let pe = env * (omega_rabi * t).cos().powi(2);
            points.push((t, pe.clamp(0.0, 1.0)));
        }

        points
    }
}
```